



Webpage

Stacks, Queues, Linked lists, Trees, Binary search trees, Binary heaps, Graphs.

Mark Distribution in Previous GATE

Year	2026 - 1	2026 - 2	2025 - 1	2025 - 2	2024 - 1	2024 - 2	2023	2022	2021 - 1	2021 - 2	Minimum
1 Mark Count	0	1	3	2	0	0	2	2	4	2	0
2 Marks Count	1	1	1	2	1	2	3	1	1	0	0
Total Marks	2	3	5	6	2	4	8	4	6	2	2

Welcome to the "Programming and Data Structures: Data Structures" chapter, a cornerstone of the GATE Computer Science syllabus. This subject is fundamental to understanding how data is organized, stored, and manipulated efficiently, which is crucial for designing optimal algorithms and software systems. For GATE CS, Data Structures typically carries a significant weightage, often contributing 8-12 marks, sometimes more, through a mix of Multiple Choice Questions (MCQs) and Numerical Answer Type (NAT) questions. Questions frequently test your understanding of core concepts, time and space complexity analysis of operations, properties of various data structures, and their applications in problem-solving. Expect problems involving tracing operations on specific data structures like AVL trees or heaps, analyzing the complexity of given code snippets, or determining the output of tree traversals.

Topic-wise Key Concepts

AVL Tree

An AVL tree is a self-balancing Binary Search Tree (BST) where the difference between the heights of the left and right subtrees for any node is at most 1. This strict balance ensures that the tree remains relatively balanced, preventing worst-case scenarios of skewed BSTs.

- **Balance Factor:** For any node N ,
 $BalanceFactor(N) = Height(RightSubtree(N)) - Height(LeftSubtree(N))$. In an AVL tree, $BalanceFactor(N) \in \{-1, 0, 1\}$.
- **Rotations:** When an insertion or deletion causes a node to become unbalanced (balance factor becomes ± 2), rotations are performed to restore balance. There are four types of rotations:
 1. **LL Rotation (Left-Left):** Occurs when an insertion into the left subtree of the left child causes imbalance. A single right rotation is performed.
 2. **RR Rotation (Right-Right):** Occurs when an insertion into the right subtree of the right child causes imbalance. A single left rotation is performed.
 3. **LR Rotation (Left-Right):** Occurs when an insertion into the right subtree of the left child causes imbalance. A left rotation on the child, followed by a right rotation on the parent.
 4. **RL Rotation (Right-Left):** Occurs when an insertion into the left subtree of the right child causes imbalance. A right rotation on the child, followed by a left rotation on the parent.
- **Height of an AVL Tree:** For an AVL tree with n nodes, its height h is always $O(\log n)$. Specifically, $h < 1.44 \log_2(n + 2)$.
- **Time Complexity:** Insertion, deletion, and search operations all take $O(\log n)$ time in the worst case due to the balanced nature.

Common Pitfalls: Incorrectly identifying the type of rotation needed, especially for LR and RL cases. Forgetting to update heights and balance factors after rotations. Tracing rotations correctly requires careful step-by-step analysis.

Problem-Solving Techniques: Practice tracing insertions/deletions and the subsequent rotations on small example AVL trees. Understand the pivot node and the child involved in the imbalance.

Abstract Data Type (ADT)

An Abstract Data Type (ADT) is a mathematical model for data types, defining the data stored and the operations that can be performed on it, without specifying how these operations are implemented. It focuses on "what" an operation does rather than "how" it does it.

- **Key Properties:**
 - **Abstraction:** Hides the internal implementation details from the user.
 - **Encapsulation:** Bundles data and methods that operate on the data into a single unit.
- **Examples:** Stack, Queue, List, Priority Queue, Map, Set are all ADTs. A data structure (e.g., array, linked list) is a

concrete implementation of an ADT.

Common Pitfalls: Confusing an ADT with a data structure. An ADT is a logical concept, while a data structure is a physical implementation.

Problem-Solving Techniques: Identify the core operations and their logical behavior when asked to define or recognize an ADT.

Array

An array is a collection of elements of the same data type, stored in contiguous memory locations. Elements are accessed using an integer index.

- **Address Calculation (1D Array):** For an array A starting at $BaseAddress$, with elements of size S , and lower bound L_0 :

$$Address(A[i]) = BaseAddress + (i - L_0) \times S$$

If $L_0 = 0$, then $Address(A[i]) = BaseAddress + i \times S$.

- **Address Calculation (2D Array - Row-Major Order):** For an array $A[R][C]$ (R rows, C columns), starting at $BaseAddress$, with element size S , and lower bounds L_{row}, L_{col} :

$$Address(A[i][j]) = BaseAddress + ((i - L_{row}) \times C + (j - L_{col})) \times S$$

If $L_{row} = 0, L_{col} = 0$, then $Address(A[i][j]) = BaseAddress + (i \times C + j) \times S$.

- **Address Calculation (2D Array - Column-Major Order):** For an array $A[R][C]$, starting at $BaseAddress$, with element size S , and lower bounds L_{row}, L_{col} :

$$Address(A[i][j]) = BaseAddress + ((j - L_{col}) \times R + (i - L_{row})) \times S$$

If $L_{row} = 0, L_{col} = 0$, then $Address(A[i][j]) = BaseAddress + (j \times R + i) \times S$.

- **Time Complexity:**
 - Accessing an element: $O(1)$
 - Insertion/Deletion at end: $O(1)$ (if space available)
 - Insertion/Deletion at beginning/middle: $O(n)$ (requires shifting elements)

Common Pitfalls: Off-by-one errors in index calculations, especially with non-zero lower bounds. Confusing row-major with column-major order.

Problem-Solving Techniques: Carefully apply the address calculation formulas. Understand the memory layout for multi-dimensional arrays.

Binary Heap

A binary heap is a complete binary tree that satisfies the heap property: for a min-heap, every parent node's value is less than or equal to its children's values; for a max-heap, every parent node's value is greater than or equal to its children's values. Heaps are typically implemented using an array.

- **Array Representation (0-indexed):**
 - Parent of node at index i : $\lfloor (i - 1) / 2 \rfloor$
 - Left child of node at index i : $2i + 1$
 - Right child of node at index i : $2i + 2$
- **Heap Properties:**
 - **Completeness:** All levels are completely filled except possibly the last level, which is filled from left to right.
 - **Heap Property:** (Min-heap or Max-heap)
- **Time Complexity:**
 - Insert: $O(\log n)$ (heapify-up)
 - Delete-min/max (extract-root): $O(\log n)$ (heapify-down)
 - Build Heap from an array of n elements: $O(n)$
 - Search: $O(n)$ (Heaps are not designed for efficient search of arbitrary elements)

Common Pitfalls: Violating the completeness property. Incorrectly applying heapify-up or heapify-down logic. Confusing min-heap and max-heap properties.

Problem-Solving Techniques: Trace heap operations (insert, delete) step-by-step. For building a heap, start from the last non-leaf node and perform heapify-down upwards.

Binary Search Tree (BST)

A Binary Search Tree (BST) is a binary tree where for every node, all values in its left subtree are less than the node's value, and all values in its right subtree are greater than the node's value. Duplicate values are typically not allowed or handled specifically (e.g., placed in the right subtree).

- **BST Properties:**

- Left subtree values $<$ Root value
- Right subtree values $>$ Root value
- Both left and right subtrees are also BSTs.
- Inorder traversal of a BST yields elements in sorted order.

- **Time Complexity (Average Case):**

- Search, Insertion, Deletion: $O(\log n)$

- **Time Complexity (Worst Case - Skewed Tree):**

- Search, Insertion, Deletion: $O(n)$ (occurs when elements are inserted in sorted or reverse-sorted order, leading to a linked list-like structure)

- **Deletion Cases:**

1. Node to be deleted is a leaf: Simply remove it.
2. Node has one child: Replace the node with its child.
3. Node has two children: Replace the node's value with its inorder predecessor (largest in left subtree) or inorder successor (smallest in right subtree), then delete the predecessor/successor node (which will have 0 or 1 child).

Common Pitfalls: Forgetting the worst-case $O(n)$ complexity for skewed trees. Incorrectly handling deletion of nodes with two children. Not understanding that inorder traversal is key to sorted output.

Problem-Solving Techniques: Practice constructing BSTs and performing deletions. Understand the recursive nature of BST operations.

Binary Tree

A binary tree is a tree data structure in which each node has at most two children, referred to as the left child and the right child.

- **Key Properties:**

- **Full Binary Tree:** Every node has either 0 or 2 children.
- **Complete Binary Tree:** All levels are completely filled except possibly the last level, which is filled from left to right.
- **Perfect Binary Tree:** All internal nodes have two children, and all leaf nodes are at the same level. (A perfect binary tree is always full and complete).
- **Skewed Binary Tree:** All nodes have only one child (either left or right), resembling a linked list.

- **Formulas:**

- Maximum number of nodes at level k (root at level 0): 2^k
- Maximum number of nodes in a binary tree of height h (max levels $h + 1$): $2^{h+1} - 1$
- Minimum height of a binary tree with n nodes: $\lceil \log_2(n + 1) \rceil - 1$ or $\lfloor \log_2 n \rfloor$ (for complete binary tree)
- Number of leaf nodes L and internal nodes I in a full binary tree: $L = I + 1$

Common Pitfalls: Confusing the definitions of full, complete, and perfect binary trees. Miscalculating height or number of nodes.

Problem-Solving Techniques: Draw examples to visualize properties. Apply formulas carefully based on the definition of the tree type.

Data Structures

Data structures are specialized formats for organizing and storing data in a computer so that it can be accessed and modified efficiently. They provide a means to manage large amounts of data effectively for various applications.

- **Classification:**

- **Primitive vs. Non-Primitive:** Primitive (int, char, float), Non-Primitive (Arrays, Linked Lists, Trees, Graphs).
- **Linear vs. Non-Linear:** Linear (Arrays, Linked Lists, Stacks, Queues), Non-Linear (Trees, Graphs).

- **Static vs. Dynamic:** Static (fixed size at compile time, e.g., arrays), Dynamic (size can change at runtime, e.g., linked lists).
- **Importance:** Choosing the right data structure is critical for optimal algorithm performance (time and space complexity).

Common Pitfalls: Not understanding the trade-offs between different data structures (e.g., array vs. linked list for insertion/deletion).

Problem-Solving Techniques: Analyze problem requirements (e.g., frequent insertions, fast searches, memory constraints) to select the most appropriate data structure.

Hashing

Hashing is a technique used to map keys to array indices (hash table slots) for efficient data storage and retrieval. A hash function $h(k)$ computes an index for a given key k .

- **Hash Functions:**
 - **Division Method:** $h(k) = k \pmod{m}$, where m is the size of the hash table (often a prime number not close to a power of 2).
 - **Multiplication Method:** $h(k) = \lfloor m(kA \pmod{1}) \rfloor$, where A is a constant $0 < A < 1$ (e.g., $A \approx (\sqrt{5} - 1)/2$).
 - **Mid-Square Method:** Square the key, then extract some middle digits.
 - **Folding Method:** Divide the key into parts, then combine them (e.g., add them).
- **Collision Resolution Techniques:** When two different keys map to the same index.
 - **Chaining:** Each hash table slot points to a linked list of all keys that hash to that slot.
 - Average case for search/insert/delete: $O(1 + \alpha)$, where α is the load factor.
 - Worst case: $O(n)$ if all keys hash to the same slot.
 - **Open Addressing:** All elements are stored directly in the hash table. When a collision occurs, probe for an alternative empty slot.
 - **Linear Probing:** $h(k, i) = (h'(k) + i) \pmod{m}$, where $i = 0, 1, 2, \dots$. Suffers from primary clustering.
 - **Quadratic Probing:** $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod{m}$. Reduces primary clustering but can suffer from secondary clustering.
 - **Double Hashing:** $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod{m}$. Uses two hash functions to generate probe sequences, reducing clustering.
- **Load Factor (α):** $\alpha = n/m$, where n is the number of elements and m is the table size.
 - For chaining: α can be greater than 1.
 - For open addressing: $\alpha \leq 1$.

Common Pitfalls: Choosing a non-prime table size for division method. Incorrectly applying probing sequences. Understanding the difference between primary and secondary clustering.

Problem-Solving Techniques: Trace insertions and searches with given hash functions and collision resolution strategies. Calculate load factor and analyze performance implications.

Infix, Prefix, Postfix Notations

These are different ways to write arithmetic expressions, differing in the position of operators relative to their operands.

- **Infix Notation:** Operator is between operands (e.g., $A + B$). This is the human-readable form.
- **Prefix Notation (Polish Notation):** Operator precedes its operands (e.g., $+AB$).
- **Postfix Notation (Reverse Polish Notation):** Operator follows its operands (e.g., $AB+$).
- **Properties:** Prefix and Postfix notations do not require parentheses to define operator precedence or associativity, making them unambiguous for computer evaluation.
- **Conversion Rules:**
 - **Infix to Postfix/Prefix:** Typically uses a stack. Operators are pushed onto the stack based on precedence and associativity. Operands are directly appended to the output.
 - **Evaluation of Postfix/Prefix:** Also uses a stack. For postfix, operands are pushed; when an operator is encountered, pop two operands, perform the operation, and push the result. For prefix, scan from right to left.

Common Pitfalls: Incorrectly handling operator precedence and associativity during conversions. Mistakes in stack operations (push/pop order).

Problem-Solving Techniques: Practice conversions using the stack-based algorithms. Remember operator

precedence (e.g., $\times, / > +, -$) and associativity (most are left-associative, exponentiation is right-associative).

Linked List

A linked list is a linear data structure where elements are not stored in contiguous memory locations. Instead, each element (node) contains data and a pointer (or link) to the next node in the sequence.

- **Types of Linked Lists:**

- **Singly Linked List:** Each node points to the next node. Traversal is unidirectional.
- **Doubly Linked List:** Each node has pointers to both the next and previous nodes. Traversal is bidirectional.
- **Circular Linked List:** The last node points back to the first node (or head). Can be singly or doubly circular.

- **Time Complexity:**

- Insertion/Deletion at beginning: $O(1)$
- Insertion/Deletion at end: $O(n)$ for singly linked list (requires traversal), $O(1)$ if tail pointer is maintained. $O(1)$ for doubly linked list if tail pointer is maintained.
- Insertion/Deletion at specific position: $O(n)$ (requires traversal to find position)
- Search: $O(n)$

- **Advantages:** Dynamic size, efficient insertions/deletions at specific points (if pointer to previous node is available).

- **Disadvantages:** More memory overhead (for pointers), no random access ($O(n)$ for access).

Common Pitfalls: Null pointer exceptions when traversing or manipulating the list. Incorrectly updating pointers during insertion/deletion, leading to lost nodes or broken links. Handling edge cases like empty list or single-node list.

Problem-Solving Techniques: Draw diagrams to visualize pointer changes. Pay close attention to the order of pointer updates. Always check for null pointers.

Priority Queue

A Priority Queue is an Abstract Data Type (ADT) that functions like a queue but with an additional concept of "priority." Elements are retrieved based on their priority, not necessarily their insertion order. The element with the highest (or lowest) priority is always dequeued first.

- **Core Operations:**

- **Insert (enqueue):** Adds an element with a given priority.
- **Extract-Min/Max (dequeue):** Removes and returns the element with the highest (or lowest) priority.
- **Peek:** Returns the highest (or lowest) priority element without removing it.

- **Implementations:**

- **Binary Heap:** Most efficient implementation, providing $O(\log n)$ for insert and extract-min/max.
- **Unsorted Array/Linked List:** $O(1)$ insert, $O(n)$ extract-min/max.
- **Sorted Array/Linked List:** $O(n)$ insert, $O(1)$ extract-min/max.

Common Pitfalls: Confusing a priority queue with a regular queue. Not understanding that the underlying data structure (e.g., heap) determines the complexity.

Problem-Solving Techniques: When a problem requires always processing the "most important" item next, a priority queue (implemented with a heap) is often the solution.

Queue

A Queue is a linear Abstract Data Type (ADT) that follows the First-In, First-Out (FIFO) principle. Elements are added at one end (rear/tail) and removed from the other end (front/head).

- **Core Operations:**

- **Enqueue:** Adds an element to the rear of the queue.
- **Dequeue:** Removes and returns the element from the front of the queue.
- **Front/Peek:** Returns the element at the front without removing it.
- **IsEmpty, IsFull:** Checks the state of the queue.

- **Implementations:**

- **Array-based:** Can lead to "queue full" even if space is available (linear array). Circular array implementation solves this.
- **Linked List-based:** More dynamic, no fixed size issues.

- **Time Complexity (Array or Linked List):** All core operations (enqueue, dequeue, front) are $O(1)$.

Common Pitfalls: Underflow (dequeuing from an empty queue) and Overflow (enqueueing into a full array-based queue). Incorrectly managing front and rear pointers in array implementations, especially circular queues.

Problem-Solving Techniques: Trace operations carefully. Understand how front and rear pointers move. For circular queues, remember the modulus operator for index calculations: $(index + 1) \pmod{Size}$.

Stack

A Stack is a linear Abstract Data Type (ADT) that follows the Last-In, First-Out (LIFO) principle. Elements are added and removed only from one end, called the "top."

- **Core Operations:**

- **Push:** Adds an element to the top of the stack.
- **Pop:** Removes and returns the element from the top of the stack.
- **Peek/Top:** Returns the element at the top without removing it.
- **IsEmpty, IsFull:** Checks the state of the stack.

- **Implementations:**

- **Array-based:** Uses a fixed-size array and a top pointer/index.
- **Linked List-based:** Each node points to the next, with the head of the list being the top of the stack.

- **Time Complexity (Array or Linked List):** All core operations (push, pop, peek) are $O(1)$.

- **Applications:** Function call stack, expression evaluation (infix to postfix/prefix conversion), undo/redo functionality, backtracking algorithms.

Common Pitfalls: Underflow (popping from an empty stack) and Overflow (pushing into a full array-based stack). Incorrectly managing the top pointer/index.

Problem-Solving Techniques: Trace stack operations. Recognize problems that naturally fit the LIFO principle (e.g., reversing a sequence, checking parenthesis balance).

Time Complexity

Time complexity measures the amount of time an algorithm takes to run as a function of the input size (n). It's typically expressed using asymptotic notations to describe the growth rate for large n .

- **Asymptotic Notations:**

- **Big-O Notation (O):** Upper bound. $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Describes the worst-case time.
- **Big-Omega Notation (Ω):** Lower bound. $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$. Describes the best-case time.
- **Big-Theta Notation (Θ):** Tight bound. $f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. Describes average-case time or when best and worst cases are the same.
- **Little-o Notation (o):** Strict upper bound. $f(n) = o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.
- **Little-omega Notation (ω):** Strict lower bound. $f(n) = \omega(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$.

- **Common Growth Rates (from fastest to slowest):**

$$O(n!), O(2^n), O(n^3), O(n^2), O(n \log n), O(n), O(\log n), O(1)$$

- **Master Theorem for Recurrence Relations:** For recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1, b > 1$ are constants, and $f(n)$ is an asymptotically positive function:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, AND if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$.

Common Pitfalls: Ignoring constant factors or lower-order terms (which are dropped in asymptotic analysis). Confusing best, worst, and average case complexities. Incorrectly applying Master Theorem conditions.

Problem-Solving Techniques: Analyze loops (number of iterations). For recursive functions, set up recurrence relations and solve them (often using Master Theorem or substitution method). Identify the dominant term in a sum of complexities.

Tree

A tree is a non-linear, hierarchical data structure consisting of nodes connected by edges. It is a connected acyclic graph, meaning there are no cycles and all nodes are reachable from the root.

- **Key Terminology:**

- **Root:** The topmost node of the tree.
- **Parent:** A node that has one or more child nodes.
- **Child:** A node directly connected to another node when moving away from the root.
- **Sibling:** Nodes that share the same parent.
- **Leaf Node (External Node):** A node with no children.
- **Internal Node:** A node with at least one child.
- **Ancestors:** All nodes on the path from the root to a node.
- **Descendants:** All nodes in the subtree rooted at a node.
- **Depth:** The length of the path from the root to a node (root is at depth 0).
- **Height:** The length of the longest path from a node to a leaf. The height of the tree is the height of its root.
- **Degree of a Node:** Number of children it has.
- **Degree of a Tree:** Maximum degree of any node in the tree.

- **Properties:**

- A tree with n nodes has exactly $n - 1$ edges.
- There is a unique path between any two nodes in a tree.

Common Pitfalls: Confusing tree terminology (e.g., depth vs. height, parent vs. child). Forgetting the $n - 1$ edges property.

Problem-Solving Techniques: Draw trees to visualize concepts. Understand recursive definitions for tree operations.

Tree Traversal

Tree traversal refers to the process of visiting each node in a tree exactly once in a systematic way. There are two main categories: Depth-First Search (DFS) and Breadth-First Search (BFS).

- **Depth-First Search (DFS) Traversal:** Uses a stack (implicitly via recursion) to explore as far as possible along each branch before backtracking.
 - **Inorder Traversal (Left, Root, Right):** Visits the left subtree, then the root, then the right subtree. For BSTs, this yields sorted elements.
 - **Preorder Traversal (Root, Left, Right):** Visits the root, then the left subtree, then the right subtree. Useful for creating a copy of the tree.
 - **Postorder Traversal (Left, Right, Root):** Visits the left subtree, then the right subtree, then the root. Useful for deleting a tree (deletes children before parent).
- **Breadth-First Search (BFS) Traversal (Level Order Traversal):** Visits nodes level by level, from left to right. Uses a queue.
 - Starts at the root, then visits all nodes at depth 1, then all nodes at depth 2, and so on.

Common Pitfalls: Incorrectly applying the order of visits for DFS traversals. Difficulty in reconstructing a tree from given traversals (e.g., Inorder + Preorder can reconstruct a unique BST).

Problem-Solving Techniques: Practice tracing all four traversal types on various trees. Remember the specific order for each. For tree reconstruction, use one traversal (e.g., Preorder) to find the root, then use another (e.g., Inorder) to partition the remaining elements into left and right subtrees.

Uniform Hashing

Uniform Hashing is an idealized assumption used in the analysis of hashing algorithms. It states that each key is equally likely to hash to any slot in the hash table, independently of where other keys hash.

- **Key Property:** If we have n keys and m slots, the probability that a key hashes to any particular slot is $1/m$.
- **Implications:**
 - Minimizes collisions in theory.
 - Leads to the best possible average-case performance for hash tables.
 - Under simple uniform hashing, the expected length of a chain in chaining is $\alpha = n/m$.
 - Under uniform hashing, the expected number of probes for unsuccessful search in open addressing is $1/(1 - \alpha)$.

Common Pitfalls: Assuming uniform hashing holds true in practice. Real-world hash functions are rarely perfectly uniform, leading to deviations from theoretical performance.

Problem-Solving Techniques: Understand that uniform hashing is a theoretical model for analyzing the average-case performance of hashing. Use its properties to calculate expected values related to collisions or probes.

Quick Formula Reference

Topic	Formula/Concept
AVL Tree	Balance Factor: $Height(Right) - Height(Left) \in \{-1, 0, 1\}$ Height: $O(\log n)$
Array (1D)	$Address(A[i]) = BaseAddress + (i - L_0) \times S$
Array (2D - Row-Major)	$Address(A[i][j]) = BaseAddress + ((i - L_{row}) \times C + (j - L_{col})) \times S$
Array (2D - Column-Major)	$Address(A[i][j]) = BaseAddress + ((j - L_{col}) \times R + (i - L_{row})) \times S$
Binary Heap (0-indexed)	Parent of i : $\lfloor (i - 1) / 2 \rfloor$ Left child of i : $2i + 1$ Right child of i : $2i + 2$ Max nodes at level k : 2^k
Binary Tree	Max nodes in height h tree: $2^{h+1} - 1$ Min height for n nodes: $\lfloor \log_2 n \rfloor$ (complete tree) Full Binary Tree: $L = I + 1$
Hashing (Division)	$h(k) = k \pmod m$
Hashing (Multiplication)	$h(k) = \lfloor m(kA \pmod 1) \rfloor$
Hashing (Linear Probing)	$h(k, i) = (h'(k) + i) \pmod m$
Hashing (Quadratic Probing)	$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod m$
Hashing (Double Hashing)	$h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod m$
Hashing (Load Factor)	$\alpha = n/m$
Time Complexity (Big-O)	$f(n) = O(g(n))$ if $f(n) \leq c \cdot g(n)$ for $n \geq n_0$
Time Complexity (Big-Omega)	$f(n) = \Omega(g(n))$ if $c \cdot g(n) \leq f(n)$ for $n \geq n_0$
Time Complexity (Big-Theta)	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for $n \geq n_0$
Master Theorem (Case 1)	If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$
Master Theorem (Case 2)	If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$
Master Theorem (Case 3)	If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$, then $T(n) = \Theta(f(n))$
Tree (General)	n nodes, $n - 1$ edges

Important Tips for GATE

- **Master Time & Space Complexity:** This is arguably the most crucial aspect. Be able to analyze the complexity of operations on all data structures and for given code snippets. Understand best, worst, and average cases. Practice applying the Master Theorem rigorously.
- **Understand ADT vs. Data Structure:** Clearly differentiate between the abstract concept (ADT) and its concrete implementation (data structure). For example, a Stack is an ADT, while an array-based stack or linked-list based stack are data structures implementing the Stack ADT.
- **Visualize Operations:** For tree-based structures (AVL, BST, Heap) and linked lists, always draw diagrams and trace operations step-by-step. This helps catch errors in pointer manipulation, rotations, or heapify processes.
- **Practice Tree Traversals:** Be proficient in Inorder, Preorder, Postorder, and Level-order traversals. Understand their applications and how to reconstruct a tree from given traversals (e.g., Preorder + Inorder).
- **Hashing Concepts are Key:** Pay close attention to different hash functions, collision resolution techniques (chaining, linear probing, quadratic probing, double hashing), and their impact on performance (load factor, clustering). Be ready to trace insertions and searches.
- **Know Standard Implementations:** Understand how basic ADTs like Stack and Queue are implemented using arrays and linked lists, including edge cases like underflow/overflow and circular array logic.
- **Identify Common Pitfalls:** Be aware of typical mistakes like off-by-one errors in array indexing, null pointer exceptions in linked lists, incorrect balance factor calculations in AVL trees, or misinterpreting operator precedence in expression conversions.
- **Practice Problem-Solving:** Data structures questions are often application-oriented. Practice a wide variety of problems to recognize when to use a particular data structure and how to apply its operations to solve the problem.

efficiently.

3.1

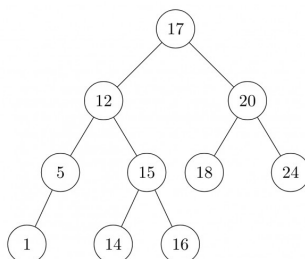
AVL Tree (6)

 **Practice Test:** Test 1 (12Q)

3.1.1 AVL Tree: GATE CSE 1988 | Question: 7ii



Mark the balance factor of each node on the tree given in the below figure and state whether it is height-balanced.



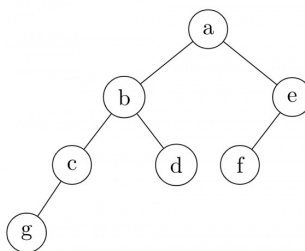
gate1988 data-structures normal descriptive avl-tree

Answer key 

3.1.2 AVL Tree: GATE CSE 1996 | Question: 1.14



In the balanced binary tree in the below figure, how many nodes will become unbalanced when a node is inserted as a child of the node “g”?



- A. 1 B. 3 C. 7 D. 8

gate1996 data-structures binary-tree avl-tree normal

Answer key 

3.1.3 AVL Tree: GATE CSE 1998 | Question: 21



- A. Derive a recurrence relation for the size of the smallest AVL tree with height h .
B. What is the size of the smallest AVL tree with height 8?

gate1998 data-structures avl-tree descriptive numerical-answers

Answer key 

3.1.4 AVL Tree: GATE CSE 2009 | Question: 37,ISRO-DEC2017-55



What is the maximum height of any AVL-tree with 7 nodes? Assume that the height of a tree with a single node is 0.

- A. 2 B. 3 C. 4 D. 5

gatecse-2009 data-structures binary-search-tree normal isrodec2017 avl-tree

Answer key 

3.1.5 AVL Tree: GATE CSE 2020 | Question: 6



What is the worst case time complexity of inserting n^2 elements into an AVL-tree with n elements initially?

- A. $\Theta(n^4)$
- B. $\Theta(n^2)$
- C. $\Theta(n^2 \log n)$
- D. $\Theta(n^3)$

gatecse-2020 binary-tree avl-tree one-mark

Answer key

3.1.6 AVL Tree: GATE IT 2008 | Question: 12



Which of the following is TRUE?

- A. The cost of searching an AVL tree is $\Theta(\log n)$ but that of a binary search tree is $O(n)$
- B. The cost of searching an AVL tree is $\Theta(\log n)$ but that of a complete binary tree is $\Theta(n \log n)$
- C. The cost of searching a binary search tree is $O(\log n)$ but that of an AVL tree is $\Theta(n)$
- D. The cost of searching an AVL tree is $\Theta(n \log n)$ but that of a binary search tree is $O(n)$

gateit-2008 data-structures binary-search-tree easy avl-tree

Answer key

3.2

Abstract Data Type (1)

3.2.1 Abstract Data Type: GATE CSE 2005 | Question: 2



An Abstract Data Type (ADT) is:

- A. same as an abstract class
- B. a data type that cannot be instantiated
- C. a data type for which only the operations defined on it can be used, but none else
- D. all of the above

gatecse-2005 data-structures normal abstract-data-type

Answer key

3.3

Array (13)

Practice Test: Test 1 (9Q)

3.3.1 Array: GATE CSE 1993 | Question: 12



The following Pascal program segments finds the largest number in a two-dimensional integer array $A[0 \dots n-1, 0 \dots n-1]$ using a single loop. Fill up the boxes to complete the program and write against **A**, **B**, **C** and **D** in your answer book Assume that max is a variable to store the largest value and i, j are the indices to the array.

```
begin
  max:=|A|, i:=0, j:=0;
  while |B| do
    begin
      if A[i, j]>max then max:=A[i, j];
      if |C| then j:=j+1;
      else begin
        j:=0;
        i:=|D|
      end
    end
  end
end
```

gate1993 data-structures array normal descriptive

Answer key

3.3.2 Array: GATE CSE 1994 | Question: 1.11



In a compact single dimensional array representation for lower triangular matrices (i.e all the elements above the diagonal are zero) of size $n \times n$, non-zero elements, (i.e elements of lower triangle) of each row are stored one after another, starting from the first row, the index of the $(i, j)^{th}$ element of the lower triangular matrix in this new representation is:

- A. $i + j$ B. $i + j - 1$ C. $(j - 1) + \frac{i(i-1)}{2}$ D. $i + \frac{j(j-1)}{2}$

gate1994 data-structures array normal

Answer key

3.3.3 Array: GATE CSE 1994 | Question: 25



An array A contains n integers in non-decreasing order, $A[1] \leq A[2] \leq \dots \leq A[n]$. Describe, using Pascal like pseudo code, a linear time algorithm to find i, j , such that $A[i] + A[j] = a$ given integer M , if such i, j exist.

gate1994 data-structures array normal descriptive

Answer key

3.3.4 Array: GATE CSE 1997 | Question: 17



An array A contains $n \geq 1$ positive integers in the locations $A[1], A[2], \dots, A[n]$. The following program fragment prints the length of a shortest sequence of consecutive elements of A , $A[i], A[i+1], \dots, A[j]$ such that the sum of their values is $\geq M$, a given positive number. It prints ' $n+1$ ' if no such sequence exists. Complete the program by filling in the boxes. In each case use the simplest possible expression. Write only the line number and the contents of the box.

```
begin
i:=1;j:=1;
sum := □
min:=n; finish:=false;
while not finish do
    if □ then
        if j=n then finish:=true
        else
            begin
                j:=j+1;
                sum:=□
            end
        else
            begin
                if (j-i) < min then min:=j-i;
                sum:=sum -A[i];
                i:=i+1;
            end
            writeln (min +1);
end.
```

gate1997 data-structures array normal descriptive

Answer key

3.3.5 Array: GATE CSE 1998 | Question: 2.14



Let A be a two dimensional array declared as follows:

A: array [1 10] [1 15] of integer;

Assuming that each integer takes one memory location, the array is stored in row-major order and the first element of the array is stored at location 100, what is the address of the element $A[i][j]$?

- A. $15i + j + 84$ B. $15j + i + 84$ C. $10i + j + 89$ D. $10j + i + 89$

gate1998 data-structures array easy

Answer key

3.3.6 Array: GATE CSE 2000 | Question: 1.2



An $n \times n$ array v is defined as follows:

$$v[i, j] = i - j \text{ for all } i, j, i \leq n, 1 \leq j \leq n$$

The sum of the elements of the array v is

- A. 0 B. $n - 1$ C. $n^2 - 3n + 2$ D. $n^2 \frac{(n+1)}{2}$

gatecse-2000 data-structures array easy

Answer key

3.3.7 Array: GATE CSE 2000 | Question: 15



Suppose you are given arrays $p[1.....N]$ and $q[1.....N]$ both uninitialized, that is, each location may contain an arbitrary value), and a variable count, initialized to 0. Consider the following procedures *set* and *is_set*:

```
set(i) {
    count = count + 1;
    q[count] = i;
    p[i] = count;
}
is_set(i) {
    if (p[i] ≤ 0 or p[i] > count)
        return false;
    if (q[p[i]] ≠ i)
        return false;
    return true;
}
```

- A. Suppose we make the following sequence of calls:
 $set(7); set(3); set(9);$
 After these sequence of calls, what is the value of count, and what do $q[1], q[2], q[3], p[7], p[3]$ and $p[9]$ contain?
- B. Complete the following statement "The first count elements of _____ contain values i such that set (_____) has been called".
- C. Show that if $set(i)$ has not been called for some i , then regardless of what $p[i]$ contains, $is_set(i)$ will return false.

gatecse-2000 data-structures array easy descriptive

Answer key

3.3.8 Array: GATE CSE 2005 | Question: 5



A program P reads in 500 integers in the range $[0, 100]$ representing the scores of 500 students. It then prints the frequency of each score above 50. What would be the best way for P to store the frequencies?

- A. An array of 50 numbers B. An array of 100 numbers
 C. An array of 500 numbers D. A dynamically allocated array of 550 numbers

gatecse-2005 data-structures array easy

Answer key

3.3.9 Array: GATE CSE 2013 | Question: 50



The procedure given below is required to find and replace certain characters inside an input character string supplied in array A . The characters to be replaced are supplied in array *oldc*, while their respective replacement characters are supplied in array *newc*. Array A has a fixed length of five characters, while arrays *oldc* and *newc* contain three characters each. However, the procedure is flawed.

```
void find_and_replace(char *A, char *oldc, char *newc) {
    for (int i=0; i<5; i++)
        for (int j=0; j<3; j++)
            if (A[i] == oldc[j])
                A[i] = newc[j];
}
```

```
}
```

The procedure is tested with the following four test cases.

1. $oldc = "abc", newc = "dab"$
2. $oldc = "cde", newc = "bcd"$
3. $oldc = "bca", newc = "cda"$
4. $oldc = "abc", newc = "bac"$

The tester now tests the program on all input strings of length five consisting of characters 'a', 'b', 'c', 'd' and 'e' with duplicates allowed. If the tester carries out this testing with the four test cases given above, how many test cases will be able to capture the flaw?

- A. Only one B. Only two C. Only three D. All four

gatecse-2013 data-structures array normal

Answer key

3.3.10 Array: GATE CSE 2013 | Question: 51



The procedure given below is required to find and replace certain characters inside an input character string supplied in array A . The characters to be replaced are supplied in array $oldc$, while their respective replacement characters are supplied in array $newc$. Array A has a fixed length of five characters, while arrays $oldc$ and $newc$ contain three characters each. However, the procedure is flawed.

```
void find_and_replace(char *A, char *oldc, char *newc) {  
    for (int i=0; i<5; i++)  
        for (int j=0; j<3; j++)  
            if (A[i] == oldc[j])  
                A[i] = newc[j];  
}
```

The procedure is tested with the following four test cases.

1. $oldc = "abc", newc = "dab"$
2. $oldc = "cde", newc = "bcd"$
3. $oldc = "bca", newc = "cda"$
4. $oldc = "abc", newc = "bac"$

If array A is made to hold the string "abcde", which of the above four test cases will be successful in exposing the flaw in this procedure?

- A. None B. 2 only C. 3 and 4 only D. 4 only

gatecse-2013 data-structures array normal

Answer key

3.3.11 Array: GATE CSE 2014 | Set 3 | Question: 42



Consider the C function given below. Assume that the array $listA$ contains $n(> 0)$ elements, sorted in ascending order.

```
int ProcessArray(int *listA, int x, int n)  
{  
    int i, j, k;  
    i = 0;    j = n-1;  
    do {  
        k = (i+j)/2;  
        if (x <= listA[k]) j = k-1;  
        if (listA[k] <= x) i = k+1;  
    }  
    while (i <= j);  
    if (listA[k] == x) return(k);  
    else return -1;  
}
```

Which one of the following statements about the function *ProcessArray* is **CORRECT**?

- A. It will run into an infinite loop when x is not in *listA*.
 B. It is an implementation of binary search.
 C. It will always find the maximum element in *listA*.
 D. It will return -1 even when x is present in *listA*.

gatecse-2014-set3 data-structures array easy

Answer key

3.3.12 Array: GATE CSE 2015 | Set 2 | Question: 31



A Young tableau is a $2D$ array of integers increasing from left to right and from top to bottom. Any unfilled entries are marked with ∞ , and hence there cannot be any entry to the right of, or below a ∞ . The following Young tableau consists of unique entries.

1	2	5	14
3	4	6	23
10	12	18	25
31	∞	∞	∞

When an element is removed from a Young tableau, other elements should be moved into its place so that the resulting table is still a Young tableau (unfilled entries may be filled with a ∞). The minimum number of entries (other than 1) to be shifted, to remove 1 from the given Young tableau is _____.

gatecse-2015-set2 databases array normal numerical-answers

Answer key 

3.3.13 Array: GATE CSE 2021 | Set 1 | Question: 2



Let P be an array containing n integers. Let t be the lowest upper bound on the number of comparisons of the array elements, required to find the minimum and maximum values in an arbitrary array of n elements. Which one of the following choices is correct?

- A. $t > 2n - 2$
 B. $t > 3\lceil \frac{n}{2} \rceil$ and $t \leq 2n - 2$
 C. $t > n$ and $t \leq 3\lceil \frac{n}{2} \rceil$
 D. $t > \lceil \log_2(n) \rceil$ and $t \leq n$

gategse-2021-set1 data-structures array one-mark

Answer key 

3.4 Binary Heap (30)

 Practice Tests: Test 1 (15Q) Test 2 (15Q) Test 3 (15Q) Test 4 (2Q)

3.4.1 Binary Heap: GATE CSE 1990 | Question: 2-viii



Match the pairs in the following questions:

(a)	A heap construction	(p)	$\Omega(n \log_{10} n)$
(b)	Constructing Hashtable with linear probing	(q)	$O(n)$
(c)	AVL tree construction	(r)	$O(n^2)$
(d)	Digital trie construction	(s)	$O(n \log_2 n)$

gate1990 match-the-following data-structures binary-heap

Answer key

3.4.2 Binary Heap: GATE CSE 1996 | Question: 2.11



The minimum number of interchanges needed to convert the array into a max-heap is

89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70

A. 0

B. 1

C. 2

D. 3

gate1996 data-structures binary-heap easy

Answer key

3.4.3 Binary Heap: GATE CSE 1999 | Question: 12



- A. In binary tree, a full node is defined to be a node with 2 children. Use induction on the height of the binary tree to prove that the number of full nodes plus one is equal to the number of leaves.
- B. Draw the min-heap that results from insertion of the following elements in order into an initially empty min-heap: 7, 6, 5, 4, 3, 2, 1. Show the result after the deletion of the root of this heap.

gate1999 data-structures binary-heap normal descriptive

Answer key

3.4.4 Binary Heap: GATE CSE 2001 | Question: 1.15



Consider any array representation of an n element binary heap where the elements are stored from index 1 to index n of the array. For the element stored at index i of the array ($i \leq n$), the index of the parent is

A. $i - 1$

B. $\lfloor \frac{i}{2} \rfloor$

C. $\lceil \frac{i}{2} \rceil$

D. $\frac{(i+1)}{2}$

gatecse-2001 data-structures binary-heap easy

Answer key

3.4.5 Binary Heap: GATE CSE 2003 | Question: 23



In a min-heap with n elements with the smallest element at the root, the 7^{th} smallest element can be found in time

A. $\Theta(n \log n)$

B. $\Theta(n)$

C. $\Theta(\log n)$

D. $\Theta(1)$

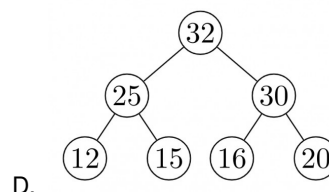
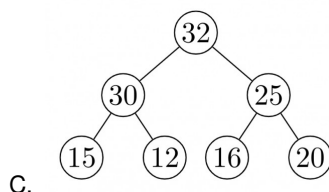
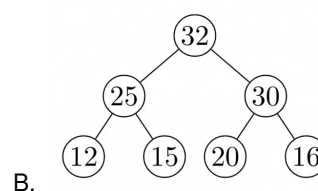
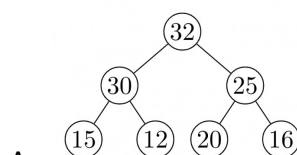
gatecse-2003 data-structures binary-heap

Answer key

3.4.6 Binary Heap: GATE CSE 2004 | Question: 37



The elements 32, 15, 20, 30, 12, 25, 16, are inserted one by one in the given order into a maxHeap. The resultant maxHeap is



gatecse-2004 data-structures binary-heap easy

Answer key

3.4.7 Binary Heap: GATE CSE 2005 | Question: 34



A priority queue is implemented as a Max-Heap. Initially, it has 5 elements. The level-order traversal of the heap is: 10, 8, 5, 3, 2. Two new elements 1 and 7 are inserted into the heap in that order. The level-order traversal of the heap after the insertion of the elements is:

- A. 10, 8, 7, 5, 3, 2, 1
- B. 10, 8, 7, 2, 3, 1, 5
- C. 10, 8, 7, 1, 2, 3, 5
- D. 10, 8, 7, 3, 2, 1, 5

gatecse-2005 data-structures binary-heap normal

Answer key

3.4.8 Binary Heap: GATE CSE 2006 | Question: 10



In a binary max heap containing n numbers, the smallest element can be found in time

- A. $O(n)$
- B. $O(\log n)$
- C. $O(\log \log n)$
- D. $O(1)$

gatecse-2006 data-structures binary-heap easy

Answer key

3.4.9 Binary Heap: GATE CSE 2006 | Question: 76



Statement for Linked Answer Questions 76 & 77:

A 3-ary max heap is like a binary max heap, but instead of 2 children, nodes have 3 children. A 3-ary heap can be represented by an array as follows: The root is stored in the first location, $a[0]$, nodes in the next level, from left to right, is stored from $a[1]$ to $a[3]$. The nodes from the second level of the tree from left to right are stored from $a[4]$ location onward. An item x can be inserted into a 3-ary heap containing n items by placing x in the location $a[n]$ and pushing it up the tree to satisfy the heap property.

Which one of the following is a valid sequence of elements in an array representing 3-ary max heap?

- A. 1, 3, 5, 6, 8, 9
- B. 9, 6, 3, 1, 8, 5
- C. 9, 3, 6, 8, 5, 1
- D. 9, 5, 6, 8, 3, 1

gatecse-2006 data-structures binary-heap normal

Answer key

3.4.10 Binary Heap: GATE CSE 2006 | Question: 77



Statement for Linked Answer Questions 76 & 77:

A 3-ary max heap is like a binary max heap, but instead of 2 children, nodes have 3 children. A 3-ary heap can be represented by an array as follows: The root is stored in the first location, $a[0]$, nodes in the next level, from left to right, is stored from $a[1]$ to $a[3]$. The nodes from the second level of the tree from left to right are stored from $a[4]$ location onward. An item x can be inserted into a 3-ary heap containing n items by placing x in the location $a[n]$ and pushing it up the tree to satisfy the heap property.

76. Which one of the following is a valid sequence of elements in an array representing 3-ary max heap?

- A. 1, 3, 5, 6, 8, 9
- B. 9, 6, 3, 1, 8, 5
- C. 9, 3, 6, 8, 5, 1
- D. 9, 5, 6, 8, 3, 1

77. Suppose the elements 7, 2, 10 and 4 are inserted, in that order, into the valid 3-ary max heap found in the previous question, Q.76. Which one of the following is the sequence of items in the array representing the resultant heap?

- A. 10, 7, 9, 8, 3, 1, 5, 2, 6, 4
- B. 10, 9, 8, 7, 6, 5, 4, 3, 2, 1
- C. 10, 9, 4, 5, 7, 6, 8, 2, 1, 3
- D. 10, 8, 6, 9, 7, 2, 3, 4, 1, 5

gatecse-2006 data-structures binary-heap normal

Answer key

3.4.11 Binary Heap: GATE CSE 2007 | Question: 47



Consider the process of inserting an element into a Max Heap, where the Max Heap is represented by an array. Suppose we perform a binary search on the path from the new leaf to the root to find the position for the newly inserted element, the number of comparisons performed is:

- A. $\Theta(\log_2 n)$
- B. $\Theta(\log_2 \log_2 n)$
- C. $\Theta(n)$
- D. $\Theta(n \log_2 n)$

gatecse-2007 data-structures binary-heap normal

Answer key

3.4.12 Binary Heap: GATE CSE 2009 | Question: 59



Consider a binary max-heap implemented using an array. Which one of the following array represents a binary max-heap?

- A. {25, 12, 16, 13, 10, 8, 14}
- B. {25, 14, 13, 16, 10, 8, 12}
- C. {25, 14, 16, 13, 10, 8, 12}
- D. {25, 14, 12, 13, 10, 8, 16}

gatecse-2009 data-structures binary-heap easy

Answer key

3.4.13 Binary Heap: GATE CSE 2009 | Question: 60



Consider a binary max-heap implemented using an array. What is the content of the array after two delete operations on {25, 14, 16, 13, 10, 8, 12}?

- A. {14, 13, 12, 10, 8}
- B. {14, 12, 13, 8, 10}
- C. {14, 13, 8, 12, 10}
- D. {14, 13, 12, 8, 10}

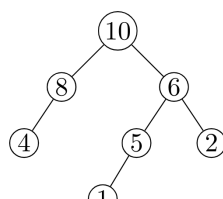
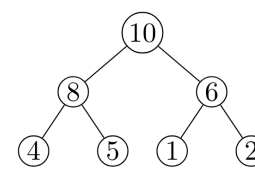
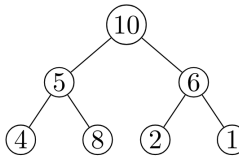
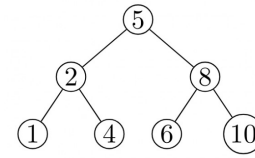
gatecse-2009 data-structures binary-heap normal

Answer key

3.4.14 Binary Heap: GATE CSE 2011 | Question: 23



A max-heap is a heap where the value of each parent is greater than or equal to the value of its children. Which of the following is a max-heap?

- A. 
- B. 
- C. 
- D. 

gatecse-2011 data-structures binary-heap easy

Answer key

3.4.15 Binary Heap: GATE CSE 2014 | Set 2 | Question: 12



A priority queue is implemented as a Max-Heap. Initially, it has 5 elements. The level-order traversal of the heap is: 10, 8, 5, 3, 2. Two new elements 1 and 7 are inserted into the heap in that order. The level-order traversal of the heap after the insertion of the elements is:

- A. 10, 8, 7, 3, 2, 1, 5
- B. 10, 8, 7, 2, 3, 1, 5
- C. 10, 8, 7, 1, 2, 3, 5
- D. 10, 8, 7, 5, 3, 2, 1

gatecse-2014-set2 data-structures binary-heap normal

Answer key

3.4.16 Binary Heap: GATE CSE 2015 | Set 1 | Question: 32



Consider a max heap, represented by the array: 40, 30, 20, 10, 15, 16, 17, 8, 4.

Array index	1	2	3	4	5	6	7	8	9
Value	40	30	20	10	15	16	17	8	4

Now consider that a value 35 is inserted into this heap. After insertion, the new heap is

- A. 40, 30, 20, 10, 15, 16, 17, 8, 4, 35
 B. 40, 35, 20, 10, 30, 16, 17, 8, 4, 15
 C. 40, 30, 20, 10, 35, 16, 17, 8, 4, 15
 D. 40, 35, 20, 10, 15, 16, 17, 8, 4, 30

gatecse-2015-set1 data-structures binary-heap easy

Answer key

3.4.17 Binary Heap: GATE CSE 2015 | Set 2 | Question: 17



Consider a complete binary tree where the left and right subtrees of the root are max-heaps. The lower bound for the number of operations to convert the tree to a heap is

- A. $\Omega(\log n)$
 B. $\Omega(n)$
 C. $\Omega(n \log n)$
 D. $\Omega(n^2)$

gatecse-2015-set2 data-structures binary-heap normal

Answer key

3.4.18 Binary Heap: GATE CSE 2015 | Set 3 | Question: 19



Consider the following array of elements.

$\langle 89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100 \rangle$

The minimum number of interchanges needed to convert it into a max-heap is

- A. 4
 B. 5
 C. 2
 D. 3

gatecse-2015-set3 data-structures binary-heap easy

Answer key

3.4.19 Binary Heap: GATE CSE 2016 | Set 1 | Question: 37



An operator $\text{delete}(i)$ for a binary heap data structure is to be designed to delete the item in the i -th node. Assume that the heap is implemented in an array and i refers to the i -th index of the array. If the heap tree has depth d (number of edges on the path from the root to the farthest leaf), then what is the time complexity to re-fix the heap efficiently after the removal of the element?

- A. $O(1)$
 B. $O(d)$ but not $O(1)$
 C. $O(2^d)$ but not $O(d)$
 D. $O(d 2^d)$ but not $O(2^d)$

gatecse-2016-set1 data-structures binary-heap normal

Answer key

3.4.20 Binary Heap: GATE CSE 2016 | Set 2 | Question: 34



A complete binary min-heap is made by including each integer in $[1, 1023]$ exactly once. The depth of a node in the heap is the length of the path from the root of the heap to that node. Thus, the root is at depth 0. The maximum depth at which integer 9 can appear is _____.

gatecse-2016-set2 data-structures binary-heap normal numerical-answers

Answer key

3.4.21 Binary Heap: GATE CSE 2018 | Question: 46



The number of possible min-heaps containing each value from $\{1, 2, 3, 4, 5, 6, 7\}$ exactly once is _____.

gatecse-2018 binary-heap numerical-answers combinatory two-marks

Answer key

3.4.22 Binary Heap: GATE CSE 2019 | Question: 40



Consider the following statements:

- I. The smallest element in a max-heap is always at a leaf node
- II. The second largest element in a max-heap is always a child of a root node
- III. A max-heap can be constructed from a binary search tree in $\Theta(n)$ time
- IV. A binary search tree can be constructed from a max-heap in $\Theta(n)$ time

Which of the above statements are TRUE?

- A. I, II and III B. I, II and IV C. I, III and IV D. II, III and IV

gatecse-2019 data-structures binary-heap two-marks

Answer key

3.4.23 Binary Heap: GATE CSE 2020 | Question: 47



Consider the array representation of a binary min-heap containing 1023 elements. The minimum number of comparisons required to find the maximum in the heap is _____.

gatecse-2020 numerical-answers binary-heap two-marks

Answer key

3.4.24 Binary Heap: GATE CSE 2021 | Set 2 | Question: 2



Let H be a binary min-heap consisting of n elements implemented as an array. What is the worst case time complexity of an optimal algorithm to find the maximum element in H ?

- A. $\Theta(1)$ B. $\Theta(\log n)$
C. $\Theta(n)$ D. $\Theta(n \log n)$

gatecse-2021-set2 data-structures binary-heap time-complexity one-mark

Answer key

3.4.25 Binary Heap: GATE CSE 2023 | Question: 2



Which one of the following sequences when stored in an array at locations $A[1], \dots, A[10]$ forms a max-heap?

- A. 23, 17, 10, 6, 13, 14, 1, 5, 7, 12 B. 23, 17, 14, 7, 13, 10, 1, 5, 6, 12
C. 23, 17, 14, 6, 13, 10, 1, 5, 7, 15 D. 23, 14, 17, 1, 10, 13, 16, 12, 7, 5

gatecse-2023 data-structures binary-heap one-mark

Answer key

3.4.26 Binary Heap: GATE CSE 2024 | Set 1 | Question: 33



Consider a binary min-heap containing 105 distinct elements. Let k be the index (in the underlying array) of the maximum element stored in the heap. The number of possible values of k is

- A. 53 B. 52 C. 27 D. 1

gatecse-2024-set1 data-structures binary-heap two-marks

Answer key

3.4.27 Binary Heap: GATE CSE 2025 | Set 1 | Question: 25



The height of any rooted tree is defined as the maximum number of edges in the path from the root node to any leaf node.

Suppose a Min-Heap T stores 32 keys. The height of T is _____. (Answer in integer)

gatecse2025-set1 data-structures binary-heap numerical-answers easy one-mark

Answer key

3.4.28 Binary Heap: GATE IT 2004 | Question: 53



An array of integers of size n can be converted into a heap by adjusting the heaps rooted at each internal node of the complete binary tree starting at the node $\lfloor (n-1)/2 \rfloor$, and doing this adjustment up to the root node (root node is at index 0) in the order $\lfloor (n-1)/2 \rfloor, \lfloor (n-3)/2 \rfloor, \dots, 0$. The time required to construct a heap in this manner is

- A. $O(\log n)$ B. $O(n)$ C. $O(n \log \log n)$ D. $O(n \log n)$

gateit-2004 data-structures binary-heap normal

Answer key

3.4.29 Binary Heap: GATE IT 2006 | Question: 44



Which of the following sequences of array elements forms a heap?

- A. $\{23, 17, 14, 6, 13, 10, 1, 12, 7, 5\}$ B. $\{23, 17, 14, 6, 13, 10, 1, 5, 7, 12\}$
C. $\{23, 17, 14, 7, 13, 10, 1, 5, 6, 12\}$ D. $\{23, 17, 14, 7, 13, 10, 1, 12, 5, 7\}$

gateit-2006 data-structures binary-heap easy

Answer key

3.4.30 Binary Heap: GATE IT 2006 | Question: 72



An array X of n distinct integers is interpreted as a complete binary tree. The index of the first element of the array is 0. If only the root node does not satisfy the heap property, the algorithm to convert the complete binary tree into a heap has the best asymptotic time complexity of

- A. $O(n)$ B. $O(\log n)$ C. $O(n \log n)$ D. $O(n \log \log n)$

gateit-2006 data-structures binary-heap easy

Answer key

3.5 Binary Search Tree (36)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(15Q\)](#) [Test 4 \(15Q\)](#)

3.5.1 Binary Search Tree: GATE CSE 1996 | Question: 2.14



A binary search tree is generated by inserting in order the following integers:

50, 15, 62, 5, 20, 58, 91, 3, 8, 37, 60, 24

The number of nodes in the left subtree and right subtree of the root respectively is

- A. (4, 7) B. (7, 4) C. (8, 3) D. (3, 8)

gate1996 data-structures binary-search-tree easy

Answer key

3.5.2 Binary Search Tree: GATE CSE 1996 | Question: 4



A binary search tree is used to locate the number 43. Which of the following probe sequences are possible and which are not? Explain.

- (a) 61 52 14 17 40 43
(b) 2 3 50 40 60 43
(c) 10 65 31 48 37 43
(d) 81 61 52 14 41 43
(e) 17 77 27 66 18 43

gate1996 data-structures binary-search-tree normal descriptive

Answer key

3.5.3 Binary Search Tree: GATE CSE 1997 | Question: 4.5



A binary search tree contains the value 1, 2, 3, 4, 5, 6, 7, 8. The tree is traversed in pre-order and the values are printed out. Which of the following sequences is a valid output?

- A. 5 3 1 2 4 7 8 6
B. 5 3 1 2 6 4 8 7
C. 5 3 2 4 1 6 7 8
D. 5 3 1 2 4 7 6 8

gate1997 data-structures binary-search-tree normal

Answer key

3.5.4 Binary Search Tree: GATE CSE 2001 | Question: 14



- A. Insert the following keys one by one into a binary search tree in the order specified.

15, 32, 20, 9, 3, 25, 12, 1

Show the final binary search tree after the insertions.

- B. Draw the binary search tree after deleting 15 from it.
C. Complete the statements $S1$, $S2$ and $S3$ in the following function so that the function computes the depth of a binary tree rooted at t .

```
typedef struct tnode{
    int key;
    struct tnode *left, *right;
} *Tree;

int depth (Tree t)
{
    int x, y;
    if (t == NULL) return 0;
    x = depth (t->left);
    S1: _____;

    S2: if (x > y) return _____;

    S3: else return _____;
}
```

gatecse-2001 data-structures binary-search-tree normal descriptive

Answer key

3.5.5 Binary Search Tree: GATE CSE 2003 | Question: 19, ISRO2009-24



Suppose the numbers 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are inserted in that order into an initially empty binary search tree. The binary search tree uses the usual ordering on natural numbers. What is the in-order traversal sequence of the resultant tree?

- A. 7 5 1 0 3 2 4 6 8 9
B. 0 2 4 3 1 6 5 9 8 7
C. 0 1 2 3 4 5 6 7 8 9
D. 9 8 6 4 2 3 0 1 5 7

gatecse-2003 binary-search-tree easy isro2009

Answer key

3.5.6 Binary Search Tree: GATE CSE 2003 | Question: 6



Let $T(n)$ be the number of different binary search trees on n distinct elements.

Then $T(n) = \sum_{k=1}^n T(k-1)T(x)$, where x is

- A. $n - k + 1$
B. $n - k$
C. $n - k - 1$
D. $n - k - 2$

gatecse-2003 normal binary-search-tree

Answer key

3.5.7 Binary Search Tree: GATE CSE 2003 | Question: 63, ISRO2009-25



A data structure is required for storing a set of integers such that each of the following operations can be done in $O(\log n)$ time, where n is the number of elements in the set.

- I. Deletion of the smallest element
- II. Insertion of an element if it is not already present in the set

Which of the following data structures can be used for this purpose?

- A. A heap can be used but not a balanced binary search tree
- B. A balanced binary search tree can be used but not a heap
- C. Both balanced binary search tree and heap can be used
- D. Neither balanced search tree nor heap can be used

gategse-2003 data-structures easy isro2009 binary-search-tree

Answer key

3.5.8 Binary Search Tree: GATE CSE 2004 | Question: 4, ISRO2009-26



The following numbers are inserted into an empty binary search tree in the given order: 10, 1, 3, 5, 15, 12, 16. What is the height of the binary search tree (the height is the maximum distance of a leaf node from the root)?

- A. 2
- B. 3
- C. 4
- D. 6

gategse-2004 data-structures binary-search-tree easy isro2009

Answer key

3.5.9 Binary Search Tree: GATE CSE 2004 | Question: 85



A program takes as input a balanced binary search tree with n leaf nodes and computes the value of a function $g(x)$ for each node x . If the cost of computing $g(x)$ is:

$$\min \left(\begin{array}{l} \text{number of leaf-nodes} \\ \text{in left-subtree of } x \end{array}, \begin{array}{l} \text{number of leaf-nodes} \\ \text{in right-subtree of } x \end{array} \right)$$

Then the worst-case time complexity of the program is?

- A. $\Theta(n)$
- B. $\Theta(n \log n)$
- C. $\Theta(n^2)$
- D. $\Theta(n^2 \log n)$

gategse-2004 binary-search-tree normal data-structures

Answer key

3.5.10 Binary Search Tree: GATE CSE 2005 | Question: 33



Postorder traversal of a given binary search tree, T produces the following sequence of keys

10, 9, 23, 22, 27, 25, 15, 50, 95, 60, 40, 29

Which one of the following sequences of keys can be the result of an in-order traversal of the tree T ?

- A. 9, 10, 15, 22, 23, 25, 27, 29, 40, 50, 60, 95
- B. 9, 10, 15, 22, 40, 50, 60, 95, 23, 25, 27, 29
- C. 29, 15, 9, 10, 25, 22, 23, 27, 40, 60, 50, 95
- D. 95, 50, 60, 40, 27, 23, 22, 25, 10, 9, 15, 29

gategse-2005 data-structures binary-search-tree easy

Answer key

3.5.11 Binary Search Tree: GATE CSE 2005 | Question: 35



How many distinct binary search trees can be created out of 4 distinct keys?

- A. 5 B. 14 C. 24 D. 42

gatecse-2005 data-structures binary-search-tree counting normal

Answer key

3.5.12 Binary Search Tree: GATE CSE 2008 | Question: 46



You are given the postorder traversal, P , of a binary search tree on the n elements $1, 2, \dots, n$. You have to determine the unique binary search tree that has P as its postorder traversal. What is the time complexity of the most efficient algorithm for doing this?

- A. $\Theta(\log n)$
B. $\Theta(n)$
C. $\Theta(n \log n)$
D. None of the above, as the tree cannot be uniquely determined

gatecse-2008 data-structures binary-search-tree normal

Answer key

3.5.13 Binary Search Tree: GATE CSE 2012 | Question: 5



The worst case running time to search for an element in a balanced binary search tree with $n2^n$ elements is

- A. $\Theta(n \log n)$ B. $\Theta(n2^n)$
C. $\Theta(n)$ D. $\Theta(\log n)$

gatecse-2012 data-structures normal binary-search-tree

Answer key

3.5.14 Binary Search Tree: GATE CSE 2013 | Question: 43



The preorder traversal sequence of a binary search tree is 30, 20, 10, 15, 25, 23, 39, 35, 42. Which one of the following is the postorder traversal sequence of the same tree?

- A. 10, 20, 15, 23, 25, 35, 42, 39, 30 B. 15, 10, 25, 23, 20, 42, 35, 39, 30
C. 15, 20, 10, 23, 25, 42, 35, 39, 30 D. 15, 10, 23, 25, 20, 35, 42, 39, 30

gatecse-2013 data-structures binary-search-tree normal

Answer key

3.5.15 Binary Search Tree: GATE CSE 2013 | Question: 7



Which one of the following is the tightest upper bound that represents the time complexity of inserting an object into a binary search tree of n nodes?

- A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

gatecse-2013 data-structures easy binary-search-tree

Answer key

3.5.16 Binary Search Tree: GATE CSE 2014 | Set 3 | Question: 39



Suppose we have a balanced binary search tree T holding n numbers. We are given two numbers L and H and wish to sum up all the numbers in T that lie between L and H . Suppose there are m such numbers in T . If the tightest upper bound on the time to compute the sum is $O(n^a \log^b n + m^c \log^d n)$, the value of $a + 10b + 100c + 1000d$ is _____.

gatecse-2014-set3 data-structures binary-search-tree numerical-answers normal

Answer key

3.5.17 Binary Search Tree: GATE CSE 2015 | Set 1 | Question: 10



Which of the following is/are correct in order traversal sequence(s) of binary search tree(s)?

- I. 3, 5, 7, 8, 15, 19, 25
- II. 5, 8, 9, 12, 10, 15, 25
- III. 2, 7, 10, 8, 14, 16, 20
- IV. 4, 6, 7, 9, 18, 20, 25

- A. I and IV only B. II and III only C. II and IV only D. II only

gatecse-2015-set1 data-structures binary-search-tree easy

Answer key

3.5.18 Binary Search Tree: GATE CSE 2015 | Set 1 | Question: 23



What are the worst-case complexities of insertion and deletion of a key in a binary search tree?

- A. $\Theta(\log n)$ for both insertion and deletion
B. $\Theta(n)$ for both insertion and deletion
C. $\Theta(n)$ for insertion and $\Theta(\log n)$ for deletion
D. $\Theta(\log n)$ for insertion and $\Theta(n)$ for deletion

gatecse-2015-set1 data-structures binary-search-tree easy

Answer key

3.5.19 Binary Search Tree: GATE CSE 2015 | Set 3 | Question: 13



While inserting the elements 71, 65, 84, 69, 67, 83 in an empty binary search tree (BST) in the sequence shown, the element in the lowest level is

- A. 65 B. 67 C. 69 D. 83

gatecse-2015-set3 data-structures binary-search-tree easy

Answer key

3.5.20 Binary Search Tree: GATE CSE 2016 | Set 2 | Question: 40



The number of ways in which the numbers 1, 2, 3, 4, 5, 6, 7 can be inserted in an empty binary search tree, such that the resulting tree has height 6, is _____.

Note: The height of a tree with a single node is 0.

gatecse-2016-set2 data-structures binary-search-tree normal numerical-answers

Answer key

3.5.21 Binary Search Tree: GATE CSE 2017 | Set 1 | Question: 6



Let T be a binary search tree with 15 nodes. The minimum and maximum possible heights of T are:

Note: The height of a tree with a single node is 0.

- A. 4 and 15 respectively. B. 3 and 14 respectively.
C. 4 and 14 respectively. D. 3 and 15 respectively.

gatecse-2017-set1 data-structures binary-search-tree easy

Answer key

3.5.22 Binary Search Tree: GATE CSE 2017 | Set 2 | Question: 36



The pre-order traversal of a binary search tree is given by 12, 8, 6, 2, 7, 9, 10, 16, 15, 19, 17, 20. Then the post-order traversal of this tree is

- A. 2, 6, 7, 8, 9, 10, 12, 15, 16, 17, 19, 20
B. 2, 7, 6, 10, 9, 8, 15, 17, 20, 19, 16, 12

- C. 7, 2, 6, 8, 9, 10, 20, 17, 19, 15, 16, 12
D. 7, 6, 2, 10, 9, 8, 15, 16, 17, 20, 19, 12

gatecse-2017-set2 data-structures binary-search-tree

Answer key

3.5.23 Binary Search Tree: GATE CSE 2020 | Question: 41



In a balanced binary search tree with n elements, what is the worst case time complexity of reporting all elements in range $[a, b]$? Assume that the number of reported elements is k .

- A. $\Theta(\log n)$ B. $\Theta(\log n + k)$
C. $\Theta(k \log n)$ D. $\Theta(n \log k)$

gatecse-2020 data-structures binary-search-tree two-marks

Answer key

3.5.24 Binary Search Tree: GATE CSE 2020 | Question: 5



The preorder traversal of a binary search tree is 15, 10, 12, 11, 20, 18, 16, 19. Which one of the following is the postorder traversal of the tree?

- A. 10, 11, 12, 15, 16, 18, 19, 20 B. 11, 12, 10, 16, 19, 18, 20, 15
C. 20, 19, 18, 16, 15, 12, 11, 10 D. 19, 16, 18, 20, 11, 12, 10, 15

gatecse-2020 binary-search-tree one-mark

Answer key

3.5.25 Binary Search Tree: GATE CSE 2021 | Set 1 | Question: 10



A binary search tree T contains n distinct elements. What is the time complexity of picking an element in T that is smaller than the maximum element in T ?

- A. $\Theta(n \log n)$ B. $\Theta(n)$ C. $\Theta(\log n)$ D. $\Theta(1)$

gatecse-2021-set1 data-structures binary-search-tree time-complexity one-mark

Answer key

3.5.26 Binary Search Tree: GATE CSE 2022 | Question: 18



Suppose a binary search tree with 1000 distinct elements is also a complete binary tree. The tree is stored using the array representation of binary heap trees. Assuming that the array indices start with 0, the 3rd largest element of the tree is stored at index _____.

gatecse-2022 numerical-answers data-structures binary-search-tree one-mark

Answer key

3.5.27 Binary Search Tree: GATE CSE 2024 | Set 2 | Question: 29



You are given a set V of distinct integers. A binary search tree T is created by inserting all elements of V one by one, starting with an empty tree. The tree T follows the convention that, at each node, all values stored in the left subtree of the node are smaller than the value stored at the node. You are not aware of the sequence in which these values were inserted into T , and you do not have access to T .

Which one of the following statements is TRUE?

- A. Inorder traversal of T can be determined from V
B. Root node of T can be determined from V
C. Preorder traversal of T can be determined from V
D. Postorder traversal of T can be determined from V

gatecse-2024-set2 binary-search-tree two-marks

Answer key

3.5.28 Binary Search Tree: GATE CSE 2025 | Set 1 | Question: 16



Which of the following statement(s) is/are TRUE for any binary search tree (BST) having n distinct integers?

- A. The maximum length of a path from the root node to any other node is $(n - 1)$.
- B. An inorder traversal will always produce a sorted sequence of elements.
- C. Finding an element takes $O(\log_2 n)$ time in the worst case.
- D. Every BST is also a Min-Heap.

gatecse2025-set1 data-structures binary-search-tree multiple-selects one-mark

Answer key

3.5.29 Binary Search Tree: GATE CSE 2025 | Set 2 | Question: 25



Suppose the values 10, -4 , 15, 30, 20, 5, 60, 19 are inserted in that order into an initially empty binary search tree. Let T be the resulting binary search tree. The number of edges in the path from the node containing 19 to the root node of T is _____. (Answer in integer)

gatecse2025-set2 data-structures binary-search-tree numerical-answers easy one-mark

Answer key

3.5.30 Binary Search Tree: GATE IT 2005 | Question: 12



The numbers $1, 2, \dots, n$ are inserted in a binary search tree in some order. In the resulting tree, the right subtree of the root contains p nodes. The first number to be inserted in the tree must be

- A. p
- B. $p + 1$
- C. $n - p$
- D. $n - p + 1$

gateit-2005 data-structures normal binary-search-tree

Answer key

3.5.31 Binary Search Tree: GATE IT 2005 | Question: 55



A binary search tree contains the numbers 1, 2, 3, 4, 5, 6, 7, 8. When the tree is traversed in pre-order and the values in each node printed out, the sequence of values obtained is 5, 3, 1, 2, 4, 6, 8, 7. If the tree is traversed in post-order, the sequence obtained would be

- A. 8, 7, 6, 5, 4, 3, 2, 1
- B. 1, 2, 3, 4, 8, 7, 6, 5
- C. 2, 1, 4, 3, 6, 7, 8, 5
- D. 2, 1, 4, 3, 7, 8, 6, 5

gateit-2005 data-structures binary-search-tree normal

Answer key

3.5.32 Binary Search Tree: GATE IT 2006 | Question: 45



Suppose that we have numbers between 1 and 100 in a binary search tree and want to search for the number 55. Which of the following sequences CANNOT be the sequence of nodes examined?

- A. {10, 75, 64, 43, 60, 57, 55}
- B. {90, 12, 68, 34, 62, 45, 55}
- C. {9, 85, 47, 68, 43, 57, 55}
- D. {79, 14, 72, 56, 16, 53, 55}

gateit-2006 data-structures binary-search-tree normal

Answer key

3.5.33 Binary Search Tree: GATE IT 2007 | Question: 29



When searching for the key value 60 in a binary search tree, nodes containing the key values 10, 20, 40, 50, 70, 80, 90 are traversed, not necessarily in the order given. How many different orders are possible in which these key values can occur on the search path from the root to the node containing the value 60?

- A. 35
- B. 64
- C. 128
- D. 5040

gateit-2007 data-structures binary-search-tree normal

Answer key

3.5.34 Binary Search Tree: GATE IT 2008 | Question: 71



A Binary Search Tree (BST) stores values in the range 37 to 573. Consider the following sequence of keys.

- I. 81, 537, 102, 439, 285, 376, 305
- II. 52, 97, 121, 195, 242, 381, 472
- III. 142, 248, 520, 386, 345, 270, 307
- IV. 550, 149, 507, 395, 463, 402, 270

Suppose the BST has been unsuccessfully searched for key 273. Which all of the above sequences list nodes in the order in which we could have encountered them in the search?

- A. II and III only B. I and III only C. III and IV only D. III only

gateit-2008 data-structures binary-search-tree normal

Answer key

3.5.35 Binary Search Tree: GATE IT 2008 | Question: 72



A Binary Search Tree (BST) stores values in the range 37 to 573. Consider the following sequence of keys.

- I. 81, 537, 102, 439, 285, 376, 305
- II. 52, 97, 121, 195, 242, 381, 472
- III. 142, 248, 520, 386, 345, 270, 307
- IV. 550, 149, 507, 395, 463, 402, 270

Which of the following statements is TRUE?

- A. I, II and IV are inorder sequences of three different BSTs
B. I is a preorder sequence of some BST with 439 as the root
C. II is an inorder sequence of some BST where 121 is the root and 52 is a leaf
D. IV is a postorder sequence of some BST with 149 as the root

gateit-2008 data-structures binary-search-tree easy

Answer key

3.5.36 Binary Search Tree: GATE IT 2008 | Question: 73



How many distinct BSTs can be constructed with 3 distinct keys?

- A. 4 B. 5 C. 6 D. 9

gateit-2008 data-structures binary-search-tree normal

Answer key

3.6 Binary Tree (53)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(15Q\)](#) [Test 3 \(15Q\)](#) [Test 4 \(6Q\)](#)

3.6.1 Binary Tree: GATE CSE 1987 | Question: 2c



State whether the following statements are TRUE or FALSE:

It is possible to construct a binary tree uniquely whose pre-order and post-order traversals are given?

gate1987 binary-tree data-structures normal true-false

Answer key

3.6.2 Binary Tree: GATE CSE 1987 | Question: 2g



State whether the following statements are TRUE or FALSE:

If the number of leaves in a tree is not a power of 2, then the tree is not a binary tree.

Answer key

3.6.3 Binary Tree: GATE CSE 1987 | Question: 7b

Construct a binary tree whose preorder traversal is

• $K L N M P R Q S T$

and inorder traversal is

• $N L K P R M S Q T$

Answer key

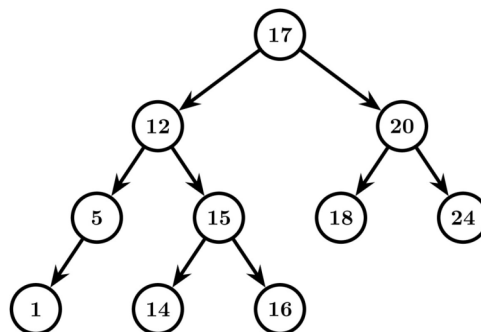
3.6.4 Binary Tree: GATE CSE 1988 | Question: 7i

Define the height of a binary tree or subtree and also define a height-balanced (AVL) tree.

Answer key

3.6.5 Binary Tree: GATE CSE 1988 | Question: 7iii

Consider the tree given in the below figure, insert 13 and show the new balance factors that would arise if the tree is not rebalanced. Finally, carry out the required rebalancing of the tree and show the new tree with the balance factors on each node.



Answer key

3.6.6 Binary Tree: GATE CSE 1989 | Question: 3-ixa

Which one of the following statements (s) is/are FALSE?

- A. Overlaying is used to run a program, which is longer than the address space of the computer.
- B. Optimal binary search tree construction can be performed efficiently by using dynamic programming.
- C. Depth first search cannot be used to find connected components of a graph.
- D. Given the prefix and postfix walks over a binary tree, the binary tree can be uniquely constructed.

Answer key

3.6.7 Binary Tree: GATE CSE 1990 | Question: 3-iv

The total external path length, EPL , of a binary tree with n external nodes is, $EPL = \sum_w I_w$, where I_w is the path length of external node w ,

- A. $\leq n^2$ always.
C. Equal to n^2 always.

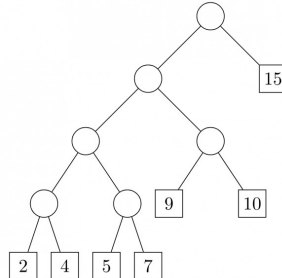
- B. $\geq n \log_2 n$ always.
D. $O(n)$ for some special trees.

gate1990 normal data-structures binary-tree multiple-selects

Answer key

3.6.8 Binary Tree: GATE CSE 1991 | Question: 01,viii

The weighted external path length of the binary tree in figure is _____

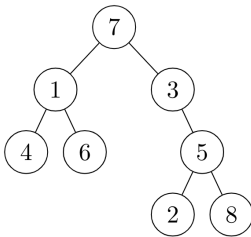


gate1991 binary-tree data-structures normal numerical-answers

Answer key

3.6.9 Binary Tree: GATE CSE 1991 | Question: 1,ix

If the binary tree in figure is traversed in inorder, then the order in which the nodes will be visited is _____

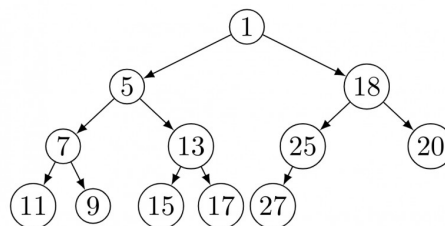


gate1991 binary-tree easy data-structures descriptive

Answer key

3.6.10 Binary Tree: GATE CSE 1991 | Question: 14,a

Consider the binary tree in the figure below:



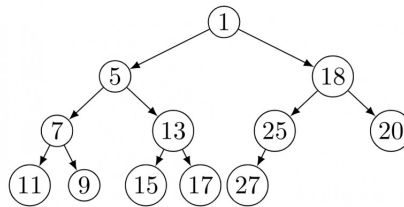
What structure is represented by the binary tree?

gate1991 data-structures binary-tree time-complexity easy descriptive

Answer key

3.6.11 Binary Tree: GATE CSE 1991 | Question: 14,b

Consider the binary tree in the figure below:



Give different steps for deleting the node with key 5 so that the structure is preserved.

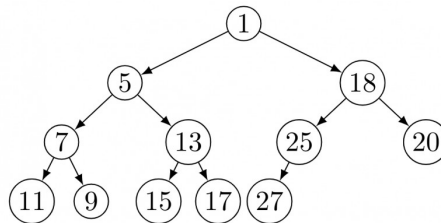
gate1991 data-structures binary-tree normal descriptive

Answer key

3.6.12 Binary Tree: GATE CSE 1991 | Question: 14,c



Consider the binary tree in the figure below:



Outline a procedure in Pseudo-code to delete an arbitrary node from such a binary tree with n nodes that preserves the structures. What is the worst-case time complexity of your procedure?

gate1991 normal data-structures binary-tree time-complexity descriptive

Answer key

3.6.13 Binary Tree: GATE CSE 1993 | Question: 16



Prove by the principal of mathematical induction that for any binary tree, in which every non-leaf node has 2- descendants, the number of leaves in the tree is one more than the number of non-leaf nodes.

gate1993 data-structures binary-tree normal descriptive

Answer key

3.6.14 Binary Tree: GATE CSE 1994 | Question: 8



A rooted tree with 12 nodes has its nodes numbered 1 to 12 in pre-order. When the tree is traversed in post-order, the nodes are visited in the order 3, 5, 4, 2, 7, 8, 6, 10, 11, 12, 9, 1.

Reconstruct the original tree from this information, that is, find the parent of each node, and show the tree diagrammatically.

gate1994 data-structures binary-tree normal descriptive

Answer key

3.6.15 Binary Tree: GATE CSE 1995 | Question: 1.17



A binary tree T has n leaf nodes. The number of nodes of degree 2 in T is

- A. $\log_2 n$ B. $n - 1$ C. n D. 2^n

gate1995 data-structures binary-tree normal

Answer key

3.6.16 Binary Tree: GATE CSE 1995 | Question: 6



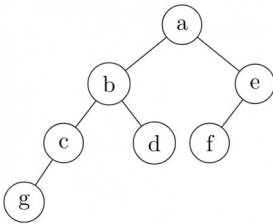
What is the number of binary trees with 3 nodes which when traversed in post-order give the sequence A, B, C ? Draw all these binary trees.

Answer key

3.6.17 Binary Tree: GATE CSE 1996 | Question: 1.15



Which of the following sequences denotes the post order traversal sequence of the below tree?

A. *f e g c d b a*C. *g c d b f e a*B. *g c b d a f e*D. *f e d g c b a*

gate1996 data-structures binary-tree easy

Answer key

3.6.18 Binary Tree: GATE CSE 1997 | Question: 16



A size-balanced binary tree is a binary tree in which for every node the difference between the number of nodes in the left and right subtree is at most 1. The distance of a node from the root is the length of the path from the root to the node. The height of a binary tree is the maximum distance of a leaf node from the root.

- A. Prove, by using induction on h , that a size-balance binary tree of height h contains at least 2^h nodes.
- B. In a size-balanced binary tree of height $h \geq 1$, how many nodes are at distance $h - 1$ from the root? Write only the answer without any explanations.

gate1997 data-structures binary-tree normal descriptive proof

Answer key

3.6.19 Binary Tree: GATE CSE 1998 | Question: 20



Draw the binary tree with node labels a, b, c, d, e, f and g for which the inorder and postorder traversals result in the following sequences:

Inorder: a f b c d g e

Postorder: a f c g e d b

gate1998 data-structures binary-tree descriptive

Answer key

3.6.20 Binary Tree: GATE CSE 2000 | Question: 1.14



Consider the following nested representation of binary trees: (XYZ) indicates Y and Z are the left and right subtrees, respectively, of node X . Note that Y and Z may be *NULL*, or further nested. Which of the following represents a valid binary tree?

A. $(1\ 2\ (4\ 5\ 6\ 7))$ C. $(1\ (2\ 3\ 4)\ (5\ 6\ 7))$ B. $(1\ (2\ 3\ 4)\ 5\ 6)\ 7)$ D. $(1\ (2\ 3\ NULL)\ (4\ 5))$

gatecse-2000 data-structures binary-tree easy

Answer key

3.6.21 Binary Tree: GATE CSE 2000 | Question: 2.16



Let LASTPOST, LASTIN and LASTPRE denote the last vertex visited in a postorder, inorder and preorder traversal respectively, of a complete binary tree. Which of the following is always true?

A. LASTIN = LASTPOST

B. LASTIN = LASTPRE

C. LASTPRE = LASTPOST

D. None of the above

gatecse-2000 data-structures binary-tree normal

Answer key

3.6.22 Binary Tree: GATE CSE 2002 | Question: 2.12



A weight-balanced tree is a binary tree in which for each node, the number of nodes in the left sub tree is at least half and at most twice the number of nodes in the right sub tree. The maximum possible height (number of nodes on the path from the root to the furthest leaf) of such a tree on n nodes is best described by which of the following?

A. $\log_2 n$

B. $\log_{\frac{4}{3}} n$

C. $\log_3 n$

D. $\log_{\frac{3}{2}} n$

gatecse-2002 data-structures binary-tree normal

Answer key

3.6.23 Binary Tree: GATE CSE 2002 | Question: 6



Draw all binary trees having exactly three nodes labeled A , B and C on which preorder traversal gives the sequence C, B, A .

gatecse-2002 data-structures binary-tree easy descriptive

Answer key

3.6.24 Binary Tree: GATE CSE 2004 | Question: 35



Consider the label sequences obtained by the following pairs of traversals on a labeled binary tree. Which of these pairs identify a tree uniquely?

- I. preorder and postorder
- II. inorder and postorder
- III. preorder and inorder
- IV. level order and postorder

A. I only

B. II, III

C. III only

D. IV only

gatecse-2004 data-structures binary-tree normal

Answer key

3.6.25 Binary Tree: GATE CSE 2004 | Question: 43



Consider the following C program segment

```
struct CellNode{
    struct CellNode *leftChild
    int element;
    struct CellNode *rightChild;
};

int Dosomething (struct CellNode *ptr)
{
    int value = 0;
    if(ptr != NULL)
    {
        if (ptr -> leftChild != NULL)
            value = 1 + Dosomething (ptr -> leftChild);
        if (ptr -> rightChild != NULL)
            value = max(value, 1 + Dosomething (ptr -> rightChild));
    }
    return(value);
}
```

The value returned by the function `DoSomething` when a pointer to the root of a non-empty tree is passed as argument is

- A. The number of leaf nodes in the tree
- C. The number of internal nodes in

- B. The number of nodes in the tree
- D. The height of the tree

the tree

gatecse-2004 data-structures binary-tree normal

Answer key

3.6.26 Binary Tree: GATE CSE 2006 | Question: 13



A scheme for storing binary trees in an array X is as follows. Indexing of X starts at 1 instead of 0. the root is stored at $X[1]$. For a node stored at $X[i]$, the left child, if any, is stored in $X[2i]$ and the right child, if any, in $X[2i + 1]$. To be able to store any binary tree on n vertices the minimum size of X should be

- A. $\log_2 n$ B. n C. $2n + 1$ D. $2^n - 1$

gatecse-2006 data-structures binary-tree normal

Answer key

3.6.27 Binary Tree: GATE CSE 2007 | Question: 12



The height of a binary tree is the maximum number of edges in any root to leaf path. The maximum number of nodes in a binary tree of height h is:

- A. $2^h - 1$ B. $2^{h-1} - 1$ C. $2^{h+1} - 1$ D. 2^{h+1}

gatecse-2007 data-structures binary-tree easy

Answer key

3.6.28 Binary Tree: GATE CSE 2007 | Question: 13



The maximum number of binary trees that can be formed with three unlabeled nodes is:

- A. 1 B. 5 C. 4 D. 3

gatecse-2007 data-structures binary-tree normal

Answer key

3.6.29 Binary Tree: GATE CSE 2007 | Question: 39, UGCNET-June2015-II: 22



The inorder and preorder traversal of a binary tree are

d b e a f c g and a b d e c f g, respectively

The postorder traversal of the binary tree is:

- A. d e b f g c a B. e d b g f c a
C. e d b f g c a D. d e f g b c a

gatecse-2007 data-structures binary-tree normal ugcnetcse-june2015-paper2

Answer key

3.6.30 Binary Tree: GATE CSE 2007 | Question: 46



Consider the following C program segment where `CellNode` represents a node in a binary tree:

```
struct CellNode {
    struct CellNode *leftChild;
    int element;
    struct CellNode *rightChild;
};

int GetValue (struct CellNode *ptr) {
    int value = 0;
    if (ptr != NULL) {
        if ((ptr->leftChild == NULL) &&
            (ptr->rightChild == NULL))
            value = 1;
        else
            value = value + GetValue(ptr->leftChild)
                + GetValue(ptr->rightChild);
    }
    return(value);
}
```

The value returned by GetValue when a pointer to the root of a binary tree is passed as its argument is:

- A. the number of nodes in the tree
- B. the number of internal nodes in the tree
- C. the number of leaf nodes in the tree
- D. the height of the tree

gatecse-2007 data-structures binary-tree normal

Answer key

3.6.31 Binary Tree: GATE CSE 2010 | Question: 10



In a binary tree with n nodes, every node has an odd number of descendants. Every node is considered to be its own descendant. What is the number of nodes in the tree that have exactly one child?

- A. 0
- B. 1
- C. $\frac{(n-1)}{2}$
- D. $n - 1$

gatecse-2010 data-structures binary-tree normal

Answer key

3.6.32 Binary Tree: GATE CSE 2011 | Question: 29



We are given a set of n distinct elements and an unlabeled binary tree with n nodes. In how many ways can we populate the tree with the given set so that it becomes a binary search tree?

- A. 0
- B. 1
- C. $n!$
- D. $\frac{1}{n+1} \cdot 2^n C_n$

gatecse-2011 binary-tree normal

Answer key

3.6.33 Binary Tree: GATE CSE 2012 | Question: 47



The height of a tree is defined as the number of edges on the longest path in the tree. The function shown in the pseudo-code below is invoked as height (root) to compute the height of a binary tree rooted at the tree pointer root.

```
int height(treeptr n)
{
    if(n == NULL) return -1;
    if(n->left == NULL)
        if(n->right == NULL) return 0;
        else return B1; // Box 1
    else
    {
        h1 = height(n->left);
        if(n->right == NULL) return (1+h1);
        else
        {
            h2 = height(n->right);
            return B2; // Box 2
        }
    }
}
```

The appropriate expressions for the two boxes **B1** and **B2** are:

- A. **B1:** $(1 + \text{height}(n \rightarrow \text{right}))$; **B2:** $(1 + \max(h1, h2))$
- B. **B1:** $(\text{height}(n \rightarrow \text{right}))$; **B2:** $(1 + \max(h1, h2))$
- C. **B1:** $\text{height}(n \rightarrow \text{right})$; **B2:** $\max(h1, h2)$
- D. **B1:** $(1 + \text{height}(n \rightarrow \text{right}))$; **B2:** $\max(h1, h2)$

gatecse-2012 data-structures binary-tree normal

Answer key

3.6.34 Binary Tree: GATE CSE 2014 | Set 1 | Question: 12



Consider a rooted n node binary tree represented using pointers. The best upper bound on the time required to determine the number of subtrees having exactly 4 nodes is $O(n^a \log^b n)$. Then the value of $a + 10b$ is _____.

gatecse-2014-set1 data-structures binary-tree numerical-answers normal

Answer key

3.6.35 Binary Tree: GATE CSE 2015 | Set 1 | Question: 25



The height of a tree is the length of the longest root-to-leaf path in it. The maximum and minimum number of nodes in a binary tree of height 5 are

- A. 63 and 6, respectively
- B. 64 and 5, respectively
- C. 32 and 6, respectively
- D. 31 and 5, respectively

gatecse-2015-set1 data-structures binary-tree easy

Answer key

3.6.36 Binary Tree: GATE CSE 2015 | Set 2 | Question: 10



A binary tree T has 20 leaves. The number of nodes in T having two children is _____.

gatecse-2015-set2 data-structures binary-tree easy numerical-answers

Answer key

3.6.37 Binary Tree: GATE CSE 2015 | Set 3 | Question: 25



Consider a binary tree T that has 200 leaf nodes. Then the number of nodes in T that have exactly two children are _____.

gatecse-2015-set3 data-structures binary-tree normal numerical-answers

Answer key

3.6.38 Binary Tree: GATE CSE 2016 | Set 2 | Question: 36



Consider the following New-order strategy for traversing a binary tree:

- Visit the root;
- Visit the right subtree using New-order;
- Visit the left subtree using New-order;

The New-order traversal of the expression tree corresponding to the reverse polish expression

$3\ 4\ * \ 5\ - \ 2\ ^ \ 6\ 7\ * \ 1\ + \ -$

is given by:

- A. $+ - 1\ 6\ 7\ * \ 2\ ^ \ 5\ - \ 3\ 4\ *$
- B. $- + 1\ * \ 6\ 7\ ^ \ 2\ - \ 5\ * \ 3\ 4$
- C. $- + 1\ * \ 7\ 6\ ^ \ 2\ - \ 5\ * \ 4\ 3$
- D. $1\ 7\ 6\ * \ + \ 2\ 5\ 4\ 3\ * \ - \ ^ \ -$

gatecse-2016-set2 data-structures binary-tree normal

Answer key

3.6.39 Binary Tree: GATE CSE 2018 | Question: 20



The postorder traversal of a binary tree is 8, 9, 6, 7, 4, 5, 2, 3, 1. The inorder traversal of the same tree is 8, 6, 9, 4, 7, 2, 5, 1, 3. The height of a tree is the length of the longest path from the root to any leaf. The height of the binary tree above is _____

gatecse-2018 data-structures binary-tree numerical-answers one-mark

Answer key

3.6.40 Binary Tree: GATE CSE 2019 | Question: 46



Let T be a full binary tree with 8 leaves. (A full binary tree has every level full.) Suppose two leaves a and b of T are chosen uniformly and independently at random. The expected value of the distance between a and b in T (ie., the number of edges in the unique path between a and b) is (rounded off to 2 decimal places) _____.

Answer key

3.6.41 Binary Tree: GATE CSE 2021 | Set 2 | Question: 16



Consider a complete binary tree with 7 nodes. Let A denote the set of first 3 elements obtained by performing Breadth-First Search (BFS) starting from the root. Let B denote the set of first 3 elements obtained by performing Depth-First Search (DFS) starting from the root.

The value of $|A - B|$ is _____

Answer key

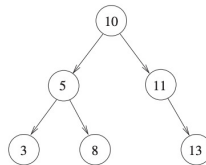
3.6.42 Binary Tree: GATE CSE 2023 | Question: 37



Consider the C function `foo` and the binary tree shown.

```
typedef struct node {
    int val;
    struct node *left, *right;
} node;

int foo(node *p) {
    int retval;
    if (p == NULL)
        return 0;
    else {
        retval = p->val + foo(p->left) + foo(p->right);
        printf("%d ", retval);
        return retval;
    }
}
```



When `foo` is called with a pointer to the root node of the given binary tree, what will it print?

- A. 3 8 5 13 11 10
 B. 3 5 8 10 11 13
 C. 3 8 16 13 24 50
 D. 3 16 8 50 24 13

Answer key

3.6.43 Binary Tree: GATE CSE 2025 | Set 2 | Question: 3



Consider a binary tree T in which every node has either zero or two children. Let $n > 0$ be the number of nodes in T .

Which ONE of the following is the number of nodes in T that have exactly two children?

- A. $\frac{n-2}{2}$
 B. $\frac{n-1}{2}$
 C. $\frac{n}{2}$
 D. $\frac{n+1}{2}$

Answer key

3.6.44 Binary Tree: GATE DS&AI 2024 | Question: 18



Consider the following tree traversals on a full binary tree:

- i. Preorder
- ii. Inorder

iii. Postorder

Which of the following traversal options is/are sufficient to uniquely reconstruct the full binary tree?

- A. (i) and (ii) B. (ii) and (iii) C. (i) and (iii) D. (ii) only

gate-ds-ai-2024 data-structures binary-tree multiple-selects one-mark

Answer key

3.6.45 Binary Tree: GATE DS&AI 2024 | Question: 42

Let H, I, L , and N represent height, number of internal nodes, number of leaf nodes, and the total number of nodes respectively in a rooted binary tree.

Which of the following statements is/are always TRUE?

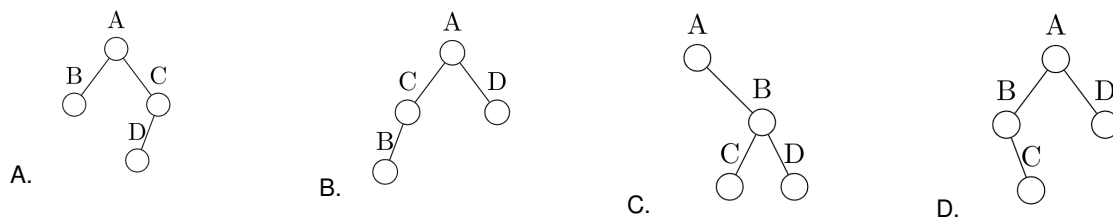
- A. $L \leq I + 1$ B. $H + 1 \leq N \leq 2^{H+1} - 1$
C. $H \leq I \leq 2^H - 1$ D. $H \leq L \leq 2^{H-1}$

gate-ds-ai-2024 data-structures binary-tree multiple-selects two-marks

Answer key

3.6.46 Binary Tree: GATE IT 2004 | Question: 54

Which one of the following binary trees has its inorder and preorder traversals as $BCAD$ and $ABCD$, respectively?



gateit-2004 binary-tree easy data-structures

Answer key

3.6.47 Binary Tree: GATE IT 2005 | Question: 50

In a binary tree, for every node the difference between the number of nodes in the left and right subtrees is at most 2. If the height of the tree is $h > 0$, then the minimum number of nodes in the tree is

- A. 2^{h-1} B. $2^{h-1} + 1$ C. $2^h - 1$ D. 2^h

gateit-2005 data-structures binary-tree normal

Answer key

3.6.48 Binary Tree: GATE IT 2006 | Question: 71

An array X of n distinct integers is interpreted as a complete binary tree. The index of the first element of the array is 0. The index of the parent of element $X[i], i \neq 0$, is?

- A. $\left\lfloor \frac{i}{2} \right\rfloor$ B. $\left\lceil \frac{i-1}{2} \right\rceil$
C. $\left\lceil \frac{i}{2} \right\rceil$ D. $\left\lceil \frac{i}{2} \right\rceil - 1$

gateit-2006 data-structures binary-tree normal

Answer key

3.6.49 Binary Tree: GATE IT 2006 | Question: 73

An array X of n distinct integers is interpreted as a complete binary tree. The index of the first element of the array is 0. If the root node is at level 0, the level of element $X[i]$, $i \neq 0$, is

- A. $\lfloor \log_2 i \rfloor$
 B. $\lceil \log_2(i+1) \rceil$
 C. $\lfloor \log_2(i+1) \rfloor$
 D. $\lceil \log_2 i \rceil$

gateit-2006 data-structures binary-tree normal

Answer key

3.6.50 Binary Tree: GATE IT 2006 | Question: 9

In a binary tree, the number of internal nodes of degree 1 is 5, and the number of internal nodes of degree 2 is 10. The number of leaf nodes in the binary tree is

- A. 10
 B. 11
 C. 12
 D. 15

gateit-2006 data-structures binary-tree normal

Answer key

3.6.51 Binary Tree: GATE IT 2008 | Question: 46

The following three are known to be the preorder, inorder and postorder sequences of a binary tree. But it is not known which is which.

- I. *MBCAFHPYK*
 II. *KAMCBYPFH*
 III. *MABCKYFPH*

Pick the true statement from the following.

- A. I and II are preorder and inorder sequences, respectively
 B. I and III are preorder and postorder sequences, respectively
 C. II is the inorder sequence, but nothing more can be said about the other two sequences
 D. II and III are the preorder and inorder sequences, respectively

gateit-2008 data-structures normal binary-tree

Answer key

3.6.52 Binary Tree: GATE IT 2008 | Question: 76

A binary tree with $n > 1$ nodes has n_1 , n_2 and n_3 nodes of degree one, two and three respectively. The degree of a node is defined as the number of its neighbours.

n_3 can be expressed as

- A. $n_1 + n_2 - 1$
 B. $n_1 - 2$
 C. $\lceil ((n_1 + n_2)/2) \rceil$
 D. $n_2 - 1$

gateit-2008 data-structures binary-tree normal

Answer key

3.6.53 Binary Tree: GATE IT 2008 | Question: 77

A binary tree with $n > 1$ nodes has n_1 , n_2 and n_3 nodes of degree one, two and three respectively. The degree of a node is defined as the number of its neighbours.

Starting with the above tree, while there remains a node v of degree two in the tree, add an edge between the two neighbours of v and then remove v from the tree. How many edges will remain at the end of the process?

- A. $2 * n_1 - 3$
 B. $n_2 + 2 * n_1 - 2$
 C. $n_3 - n_2$
 D. $n_2 + n_1 - 2$

gateit-2008 data-structures binary-tree normal

Answer key

 **Practice Tests:** [Test 1 \(15Q\)](#) [Test 2 \(6Q\)](#)

3.7.1 Data Structures: GATE CSE 1997 | Question: 6.2



Let G be the graph with 100 vertices numbered 1 to 100. Two vertices i and j are adjacent if $|i - j| = 8$ or $|i - j| = 12$. The number of connected components in G is

- A. 8 B. 4 C. 12 D. 25

gate1997 data-structures normal graph-theory

Answer key 

3.7.2 Data Structures: GATE CSE 2014 | Set 1 | Question: 3



Let $G = (V, E)$ be a directed graph where V is the set of vertices and E the set of edges. Then which one of the following graphs has the same strongly connected components as G ?

- A. $G_1 = (V, E_1)$ where $E_1 = \{(u, v) \mid (u, v) \notin E\}$
 B. $G_2 = (V, E_2)$ where $E_2 = \{(u, v) \mid (v, u) \in E\}$
 C. $G_3 = (V, E_3)$ where $E_3 = \{(u, v) \mid \text{there is a path of length } \leq 2 \text{ from } u \text{ to } v \text{ in } E\}$
 D. $G_4 = (V_4, E)$ where V_4 is the set of vertices in G which are not isolated

gatecse-2014-set1 data-structures graph-theory ambiguous

Answer key 

3.7.3 Data Structures: GATE CSE 2016 | Set 1 | Question: 38



Consider the weighted undirected graph with 4 vertices, where the weight of edge $\{i, j\}$ is given by the entry W_{ij} in the matrix W .

$$W = \begin{bmatrix} 0 & 2 & 8 & 5 \\ 2 & 0 & 5 & 8 \\ 8 & 5 & 0 & x \\ 5 & 8 & x & 0 \end{bmatrix}$$

The largest possible integer value of x , for which at least one shortest path between some pair of vertices will contain the edge with weight x is _____.

gatecse-2016-set1 data-structures graph-theory normal numerical-answers

Answer key 

3.7.4 Data Structures: GATE DS&AI 2024 | Question: 6



Match the items in **Column 1** with the items in **Column 2** in the following table:

Column 1	Column 2
(p) First In First Out	(i) Stacks
(q) Lookup Operation	(ii) Queues
(r) Last In First Out	(iii) Hash Tables

- A. (p) – (ii), (q) – (iii), (r) – (i)
 B. (p) – (ii), (q) – (i), (r) – (iii)
 C. (p) – (i), (q) – (ii), (r) – (iii)
 D. (p) – (i), (q) – (iii), (r) – (ii)

gate-ds-ai-2024 data-structures match-the-following one-mark

Practice Test: Test 1 (14Q)

3.8.1 Hashing: GATE CSE 1996 | Question: 1.13



An advantage of chained hash table (external hashing) over the open addressing scheme is

- A. Worst case complexity of search operations is less
- B. Space used is less
- C. Deletion is easier
- D. None of the above

gate1996 data-structures hashing normal

3.8.2 Hashing: GATE CSE 1996 | Question: 15



Insert the characters of the string $K R P C S N Y T J M$ into a hash table of size 10.

Use the hash function

$$h(x) = (\text{ord}(x) - \text{ord}("a") + 1) \mod 10$$

and linear probing to resolve collisions.

- A. Which insertions cause collisions?
- B. Display the final hash table.

gate1996 data-structures hashing normal descriptive

3.8.3 Hashing: GATE CSE 1997 | Question: 12



Consider a hash table with n buckets, where external (overflow) chaining is used to resolve collisions. The hash function is such that the probability that a key value is hashed to a particular bucket is $\frac{1}{n}$. The hash table is initially empty and K distinct values are inserted in the table.

- A. What is the probability that bucket number 1 is empty after the K^{th} insertion?
- B. What is the probability that no collision has occurred in any of the K insertions?
- C. What is the probability that the first collision occurs at the K^{th} insertion?

gate1997 data-structures hashing probability normal descriptive

3.8.4 Hashing: GATE CSE 2004 | Question: 7



Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function $x \mod 10$, which of the following statements are true?

- I. 9679, 1989, 4199 hash to the same value
- II. 1471, 6171 hash to the same value
- III. All elements hash to the same value
- IV. Each element hashes to a different value

- A. I only
- B. II only
- C. I and II only
- D. III or IV

gatecse-2004 data-structures hashing easy

3.8.5 Hashing: GATE CSE 2007 | Question: 40



Consider a hash table of size seven, with starting index zero, and a hash function $(3x + 4) \bmod 7$. Assuming the hash table is initially empty, which of the following is the contents of the table when the sequence 1, 3, 8, 10 is inserted into the table using closed hashing? Note that – denotes an empty location in the table.

- A. 8, –, –, –, –, –, 10
 B. 1, 8, 10, –, –, –, 3
 C. 1, –, –, –, –, –, 3
 D. 1, 10, 8, –, –, –, 3

gatecse-2007 data-structures hashing easy

Answer key

3.8.6 Hashing: GATE CSE 2009 | Question: 36



The keys 12, 18, 13, 2, 3, 23, 5 and 15 are inserted into an initially empty hash table of length 10 using open addressing with hash function $h(k) = k \bmod 10$ and linear probing. What is the resultant hash table?

- A.

0	
1	
2	2
3	23
4	
5	15
6	
7	
8	18
9	
- B.

0	
1	
2	12
3	13
4	
5	5
6	
7	
8	18
9	
- C.

0	
1	
2	12
3	13
4	2
5	3
6	23
7	5
8	18
9	15
- D.

0	
1	
2	2, 12
3	13, 3, 23
4	
5	5, 15
6	
7	
8	18
9	

gatecse-2009 data-structures hashing normal

Answer key

3.8.7 Hashing: GATE CSE 2010 | Question: 52



A hash table of length 10 uses open addressing with hash function $h(k) = k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is shown as below

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

- A. 46, 42, 34, 52, 23, 33
 B. 34, 42, 23, 52, 33, 46
 C. 46, 34, 42, 23, 52, 33
 D. 42, 46, 33, 23, 34, 52

gatecse-2010 data-structures hashing normal

Answer key

3.8.8 Hashing: GATE CSE 2010 | Question: 53



A hash table of length 10 uses open addressing with hash function $h(k) = k \bmod 10$, and linear probing. After inserting 6 values into an empty hash table, the table is shown as below

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

How many different insertion sequences of the key values using the same hash function and linear probing will result in the hash table shown above?

- A. 10 B. 20 C. 30 D. 40

data-structures hashing normal gatecse-2010

Answer key

3.8.9 Hashing: GATE CSE 2014 | Set 1 | Question: 40



Consider a hash table with 9 slots. The hash function is $h(k) = k \bmod 9$. The collisions are resolved by chaining. The following 9 keys are inserted in the order: 5, 28, 19, 15, 20, 33, 12, 17, 10. The maximum, minimum, and average chain lengths in the hash table, respectively, are

- A. 3, 0, and 1 B. 3, 3, and 3 C. 4, 0, and 1 D. 3, 0, and 2

gatecse-2014-set1 data-structures hashing normal

Answer key

3.8.10 Hashing: GATE CSE 2014 | Set 3 | Question: 40



Consider a hash table with 100 slots. Collisions are resolved using chaining. Assuming simple uniform hashing, what is the probability that the first 3 slots are unfilled after the first 3 insertions?

- A. $(97 \times 97 \times 97)/100^3$ B. $(99 \times 98 \times 97)/100^3$
C. $(97 \times 96 \times 95)/100^3$ D. $(97 \times 96 \times 95)/(3! \times 100^3)$

gatecse-2014-set3 data-structures hashing probability normal

Answer key

3.8.11 Hashing: GATE CSE 2015 | Set 2 | Question: 33



Which one of the following hash functions on integers will distribute keys most uniformly over 10 buckets numbered 0 to 9 for i ranging from 0 to 2020?

- A. $h(i) = i^2 \bmod 10$ B. $h(i) = i^3 \bmod 10$
C. $h(i) = (11 * i^2) \bmod 10$ D. $h(i) = (12 * i^2) \bmod 10$

gatecse-2015-set2 data-structures hashing normal

Answer key

3.8.12 Hashing: GATE CSE 2015 | Set 3 | Question: 17



Given that hash table T with 25 slots that stores 2000 elements, the load factor α for T is _____.

gatecse-2015-set3 data-structures hashing easy numerical-answers

Answer key

3.8.13 Hashing: GATE IT 2006 | Question: 20



Which of the following statement(s) is TRUE?

- I. A hash function takes a message of arbitrary length and generates a fixed length code.
- II. A hash function takes a message of fixed length and generates a code of variable length.
- III. A hash function may give the same hash value for distinct messages.

A. I only B. II and III only C. I and III only D. II only

gateit-2006 data-structures hashing normal

Answer key

3.8.14 Hashing: GATE IT 2007 | Question: 28



Consider a hash function that distributes keys uniformly. The hash table size is 20. After hashing of how many keys will the probability that any new key hashed collides with an existing one exceed 0.5.

A. 5 B. 6 C. 7 D. 10

gateit-2007 data-structures hashing probability normal

Answer key

3.8.15 Hashing: GATE IT 2008 | Question: 48



Consider a hash table of size 11 that uses open addressing with linear probing. Let $h(k) = k \bmod 11$ be the hash function used. A sequence of records with keys

43 36 92 87 11 4 71 13 14

is inserted into an initially empty hash table, the bins of which are indexed from zero to ten. What is the index of the bin into which the last record is inserted?

A. 3 B. 4 C. 6 D. 7

gateit-2008 data-structures hashing normal

Answer key

3.9 Infix Prefix (4)

Practice Test: Test 1 (6Q)

3.9.1 Infix Prefix: GATE CSE 1997 | Question: 1.7



Which of the following is essential for converting an infix expression to the postfix form efficiently?

- A. An operator stack
- B. An operand stack
- C. An operand stack and an operator stack
- D. A parse tree

gate1997 normal infix-prefix stack data-structures

Answer key

3.9.2 Infix Prefix: GATE CSE 1998 | Question: 19b



Compute the post fix equivalent of the following expression $3^* \log(x + 1) - \frac{a}{2}$

gate1998 stack infix-prefix descriptive

Answer key

3.9.3 Infix Prefix: GATE CSE 2004 | Question: 38, ISRO2009-27



Assume that the operators $+$, $-$, $\times$ are left associative and $^$ is right associative. The order of precedence (from highest to lowest) is $^$, $\times$, $+$, $-$. The postfix expression corresponding to the infix expression $a + b \times c - d ^ e ^ f$ is

$d_1, d_2, \dots, d_j, d, \dots, d_n$ in order without using the header.

gate1993 data-structures linked-list normal descriptive

Answer key

3.10.4 Linked List: GATE CSE 1994 | Question: 1.17, UGCNET-Sep2013-II: 32



Linked lists are not suitable data structures for which one of the following problems?

- A. Insertion sort
- B. Binary search
- C. Radix sort
- D. Polynomial manipulation

gate1994 data-structures linked-list normal ugcnetsep2013ii

Answer key

3.10.5 Linked List: GATE CSE 1995 | Question: 2.22



Which of the following statements is true?

- I. As the number of entries in a hash table increases, the number of collisions increases.
- II. Recursive programs are efficient
- III. The worst case complexity for Quicksort is $O(n^2)$
- IV. Binary search using a linear linked list is efficient

- A. I and II
- B. II and III
- C. I and IV
- D. I and III

gate1995 data-structures linked-list hashing

Answer key

3.10.6 Linked List: GATE CSE 1997 | Question: 1.4



The concatenation of two lists is to be performed on $O(1)$ time. Which of the following implementations of a list should be used?

- A. Singly linked list
- B. Doubly linked list
- C. Circular doubly linked list
- D. Array implementation of list

gate1997 data-structures linked-list easy

Answer key

3.10.7 Linked List: GATE CSE 1997 | Question: 18



Consider the following piece of 'C' code fragment that removes duplicates from an ordered list of integers.

```
Node *remove-duplicates (Node* head, int *j)
{
    Node *t1, *t2; *j=0;
    t1 = head;
    if (t1 != NULL)
        t2 = t1 ->next;
    else return head;
    *j = 1;
    if(t2 == NULL) return head;
    while (t2 != NULL)
    {
        if (t1.val != t2.val) -----> (S1)
        {
            (*j)++;
            t1 -> next = t2;
            t1 = t2; -----> (S2)
        }
        t2 = t2 ->next;
    }
    t1 -> next = NULL;
    return head;
}
```

Assume the list contains n elements ($n \geq 2$) in the following questions.

- How many times is the comparison in statement $S1$ made?
- What is the minimum and the maximum number of times statements marked $S2$ get executed?
- What is the significance of the value in the integer pointed to by j when the function completes?

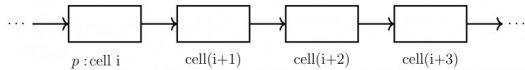
gate1997 data-structures linked-list normal descriptive

Answer key

3.10.8 Linked List: GATE CSE 1998 | Question: 19a



Let p be a pointer as shown in the figure in a single linked list.



What do the following assignment statements achieve?

```

q := p -> next
p -> next := q -> next
q -> next := (q -> next) -> next
(p -> next) -> next := q
  
```

gate1998 data-structures linked-list normal descriptive

Answer key

3.10.9 Linked List: GATE CSE 1999 | Question: 11b



Write a constant time algorithm to insert a node with data D just before the node with address p of a singly linked list.

gate1999 data-structures linked-list descriptive

Answer key

3.10.10 Linked List: GATE CSE 2002 | Question: 1.5



In the worst case, the number of comparisons needed to search a single linked list of length n for a given element is

- A. $\log n$ B. $\frac{n}{2}$ C. $\log_2 n - 1$ D. n

gatecse-2002 easy data-structures linked-list

Answer key

3.10.11 Linked List: GATE CSE 2003 | Question: 90



Consider the function f defined below.

```

struct item {
    int data;
    struct item * next;
};
int f(struct item *p) {
    return ((p == NULL) || (p->next == NULL) ||
        ((p->data <= p->next->data) &&
        f(p->next)));
}
  
```

For a given linked list p , the function f returns 1 if and only if

- the list is empty or has exactly one element
- the elements in the list are sorted in non-decreasing order of data value
- the elements in the list are sorted in non-increasing order of data value
- not all elements in the list have the same data value

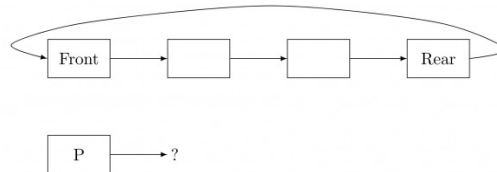
gatecse-2003 data-structures linked-list normal

Answer key

3.10.12 Linked List: GATE CSE 2004 | Question: 36



A circularly linked list is used to represent a Queue. A single variable p is used to access the Queue. To which node should p point such that both the operations **enQueue** and **deQueue** can be performed in constant time?



- A. rear node
- B. front node
- C. not possible with a single pointer
- D. node next to front

gategse-2004 data-structures linked-list normal

Answer key

3.10.13 Linked List: GATE CSE 2004 | Question: 40



Suppose each set is represented as a linked list with elements in arbitrary order. Which of the operations among **union**, **intersection**, **membership**, **cardinality** will be the slowest?

- A. union only
- B. intersection, membership
- C. membership, cardinality
- D. union, intersection

gategse-2004 data-structures linked-list normal

Answer key

3.10.14 Linked List: GATE CSE 2008 | Question: 62



The following C function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after function completes execution?

```
struct node {
    int value;
    struct node *next;
};

void rearrange(struct node *list) {
    struct node *p, *q;
    int temp;
    if (!list || !list -> next) return;
    p = list; q = list -> next;
    while(q) {
        temp = p -> value; p -> value = q -> value;
        q -> value = temp; p = q -> next;
        q = p ? p -> next : 0;
    }
}
```

- A. 1,2,3,4,5,6,7
- B. 2,1,4,3,6,5,7
- C. 1,3,2,5,4,7,6
- D. 2,3,4,5,6,7,1

gategse-2008 data-structures linked-list normal

Answer key

3.10.15 Linked List: GATE CSE 2010 | Question: 36



The following C function takes a singly-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

```
typedef struct node
{
    int value;
    struct node *next;
} Node;
Node *move_to-front(Node *head)
```

```

{
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL))
        return head;
    q = NULL;
    p = head;
    while (p->next != NULL)
    {
        q=p;
        p=p->next;
    }

    _____

    return head;
}

```

Choose the correct alternative to replace the blank line.

- A. $q = \text{NULL}; p \rightarrow \text{next} = \text{head}; \text{head} = p;$
- B. $q \rightarrow \text{next} = \text{NULL}; \text{head} = p; p \rightarrow \text{next} = \text{head};$
- C. $\text{head} = p; p \rightarrow \text{next} = q; q \rightarrow \text{next} = \text{NULL};$
- D. $q \rightarrow \text{next} = \text{NULL}; p \rightarrow \text{next} = \text{head}; \text{head} = p;$

gategse-2010 data-structures linked-list normal

Answer key

3.10.16 Linked List: GATE CSE 2016 | Set 2 | Question: 15



N items are stored in a sorted doubly linked list. For a *delete* operation, a pointer is provided to the record to be deleted. For a *decrease-key* operation, a pointer is provided to the record on which the operation is to be performed.

An algorithm performs the following operations on the list in this order: $\Theta(N)$ *delete*, $O(\log N)$ *insert*, $O(\log N)$ *find*, and $\Theta(N)$ *decrease-key*. What is the time complexity of all these operations put together?

- A. $O(\log^2 N)$
- B. $O(N)$
- C. $O(N^2)$
- D. $\Theta(N^2 \log N)$

gategse-2016-set2 data-structures linked-list time-complexity normal algorithms

Answer key

3.10.17 Linked List: GATE CSE 2017 | Set 1 | Question: 08



Consider the C code fragment given below.

```

typedef struct node {
    int data;
    node* next;
} node;

void join(node* m, node* n) {
    node* p = n;
    while (p->next != NULL) {
        p = p->next;
    }
    p->next = m;
}

```

Assuming that m and n point to valid NULL-terminated linked lists, invocation of `join` will

- A. append list m to the end of list n for all inputs.
- B. either cause a null pointer dereference or append list m to the end of list n .
- C. cause a null pointer dereference for all inputs.
- D. append list n to the end of list m for all inputs.

gategse-2017-set1 data-structures linked-list normal

Answer key

3.10.18 Linked List: GATE CSE 2020 | Question: 16



What is the worst case time complexity of inserting n elements into an empty linked list, if the linked list needs to be maintained in sorted order?

- A. $\Theta(n)$ B. $\Theta(n \log n)$ C. $\Theta(n^2)$ D. $\Theta(1)$

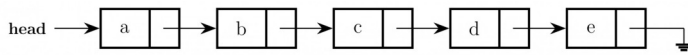
gatescse-2020 linked-list one-mark

Answer key

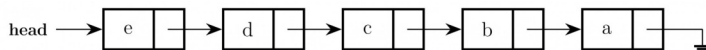
3.10.19 Linked List: GATE CSE 2022 | Question: 5



Consider the problem of reversing a singly linked list. To take an example, given the linked list below,



the reversed linked list should look like



Which one of the following statements is **TRUE** about the time complexity of algorithms that solve the above problem in $O(1)$ space?

- A. The best algorithm for the problem takes $\theta(n)$ time in the worst case.
 B. The best algorithm for the problem takes $\theta(n \log n)$ time in the worst case.
 C. The best algorithm for the problem takes $\theta(n^2)$ time in the worst case.
 D. It is not possible to reverse a singly linked list in $O(1)$ space.

gatescse-2022 data-structures linked-list one-mark

Answer key

3.10.20 Linked List: GATE CSE 2023 | Question: 3



Let **SLLdel** be a function that deletes a node in a singly-linked list given a pointer to the node and a pointer to the head of the list. Similarly, let **DLLdel** be another function that deletes a node in a doubly-linked list given a pointer to the node and a pointer to the head of the list.

Let n denote the number of nodes in each of the linked lists. Which one of the following choices is **TRUE** about the worst-case time complexity of **SLLdel** and **DLLdel**?

- A. **SLLdel** is $O(1)$ and **DLLdel** is $O(n)$ B. Both **SLLdel** and **DLLdel** are $O(\log(n))$
 C. Both **SLLdel** and **DLLdel** are $O(1)$ D. **SLLdel** is $O(n)$ and **DLLdel** is $O(1)$

gatescse-2023 data-structures linked-list one-mark

Answer key

3.10.21 Linked List: GATE CSE 2025 | Set 1 | Question: 52



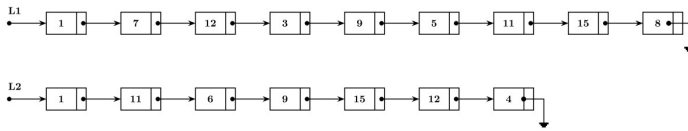
Let **LIST** be a datatype for an implementation of linked list defined as follows:

```

typedef struct list {
    int data;
    struct list *next;
} LIST;
  
```

Suppose a program has created two linked lists, $L1$ and $L2$, whose contents are given in the figure below (code for creating $L1$ and $L2$ is not provided here). $L1$ contains 9 nodes, and $L2$ contains 7 nodes.

Consider the following C program segment that modifies the list $L1$. The number of nodes that will be there in $L1$ after the execution of the code segment is _____. (Answer in integer)



```

int find (int query, LIST *list) {
    while (list != NULL) {
        if(list->data == query) return 1 ;
        list = list->next;
    }
    return 0 ;
}

int main (){
    ...
    ptr1=L1; ptr2=L2;
    while (ptr1->next != NULL){
        query = ptr1->next->data;
        if (find (query, L2))
            ptr1->next = ptr1->next->next;
        else ptr1 = ptr1->next;
    }
    ...
    return 0;
}
  
```

gatecse2025-set1 data-structures linked-list numerical-answers two-marks

Answer key

3.10.22 Linked List: GATE CSE 2026 | Set 1 | Question: 29



Consider the following code snippet in C language that computes the number of nodes in a non-empty singly linked list pointed to by the pointer variable head.

```

struct node{
    int elt;
    struct node *next;
};
int getListSize (struct node *head)
{
    if( E1 ) return 1;
    return E2;
}
  
```

Which one of the following options gives the correct replacements for the expressions **E1** and **E2**?

- A. E1: head == NULL
E2: 1 + getListSize(head)
- B. E1: head->next == NULL
E2: 1 + getListSize(head->next)
- C. E1: head == NULL
E2: 1 + getListSize(head->next)
- D. E1: head->next == NULL
E2: 1 + getListSize(head)

gatecse-2026-set1 data-structures two-marks linked-list

Answer key

3.10.23 Linked List: GATE IT 2004 | Question: 13



Let P be a singly linked list. Let Q be the pointer to an intermediate node x in the list. What is the worst-case time complexity of the best-known algorithm to delete the node x from the list ?

- A. $O(n)$
- B. $O(\log^2 n)$
- C. $O(\log n)$
- D. $O(1)$

gateit-2004 data-structures linked-list normal ambiguous

Answer key

3.10.24 Linked List: GATE IT 2005 | Question: 54



The following C function takes a singly-linked list of integers as a parameter and rearranges the elements of the list. The list is represented as pointer to a structure. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node {int value; struct node *next;};
void rearrange (struct node *list) {
    struct node *p, *q;
    int temp;
    if (!list || !list->next) return;
    p = list; q = list->next;
    while (q) {
        temp = p->value;
        p->value = q->value;
        q->value = temp;
        p = q->next;
        q = p ? p->next : 0;
    }
}
```

- A. 1,2,3,4,5,6,7
 B. 2,1,4,3,6,5,7
 C. 1,3,2,5,4,7,6
 D. 2,3,4,5,6,7,1

gateit-2005 data-structures linked-list normal

Answer key

3.11

Priority Queue (2)

3.11.1 Priority Queue: GATE CSE 1997 | Question: 4.7



A priority queue Q is used to implement a stack that stores characters. PUSH (C) is implemented as INSERT (Q, C, K) where K is an appropriate integer key chosen by the implementation. POP is implemented as DELETMIN(Q). For a sequence of operations, the keys chosen are in

- A. non-increasing order
 B. non-decreasing order
 C. strictly increasing order
 D. strictly decreasing order

gate1997 data-structures stack normal priority-queue

Answer key

3.11.2 Priority Queue: GATE CSE 2023 | Question: 36



Let A be a priority queue for maintaining a set of elements. Suppose A is implemented using a max-heap data structure. The operation EXTRACT-MAX(A) extracts and deletes the maximum element from A . The operation INSERT(A, key) inserts a new element key in A . The properties of a max-heap are preserved at the end of each of these operations.

When A contains n elements, which one of the following statements about the worst case running time of these two operations is TRUE?

- A. Both EXTRACT-MAX(A) and INSERT(A, key) run in $O(1)$.
 B. Both EXTRACT-MAX(A) and INSERT(A, key) run in $O(\log(n))$.
 C. EXTRACT-MAX(A) runs in $O(1)$ whereas INSERT(A, key) runs in $O(n)$.
 D. EXTRACT-MAX(A) runs in $O(1)$ whereas INSERT(A, key) runs in $O(\log(n))$.

gatecse-2023 data-structures priority-queue time-complexity binary-heap two-marks

Answer key

3.12

Queue (15)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(6Q\)](#)

3.12.1 Queue: GATE CSE 1992 | Question: 09



Suggest a data structure for representing a subset S of integers from 1 to n . Following operations on the set S are to be performed in constant time (independent of cardinality of S).

- i. MEMBER (X) : Check whether X is in the set S or not
- ii. FIND-ONE (S) : If S is not empty, return one element of the set S
(any arbitrary element will do)
- iii. ADD (X) : Add integer X to set S
- ii. DELETE (X) : Delete integer X from S

Give pictorial examples of your data structure. Give routines for these operations in an English like language. You may assume that the data structure has been suitable initialized. Clearly state your assumptions regarding initialization.

gate1992 data-structures normal descriptive queue

Answer key

3.12.2 Queue: GATE CSE 1994 | Question: 26



A queue Q containing n items and an empty stack S are given. It is required to transfer all the items from the queue to the stack, so that the item at the front of queue is on the TOP of the stack, and the order of all other items are preserved. Show how this can be done in $O(n)$ time using only a constant amount of additional storage. Note that the only operations which can be performed on the queue and stack are Delete, Insert, Push and Pop. Do not assume any implementation of the queue or stack.

gate1994 data-structures queue stack normal descriptive

Answer key

3.12.3 Queue: GATE CSE 1996 | Question: 1.12



Consider the following statements:

- i. First-in-first out types of computations are efficiently supported by STACKS.
- ii. Implementing LISTS on linked lists is more efficient than implementing LISTS on an array for almost all the basic LIST operations.
- iii. Implementing QUEUES on a circular array is more efficient than implementing QUEUES on a linear array with two indices.
- iv. Last-in-first-out type of computations are efficiently supported by QUEUES.

- | | |
|----------------------------|---------------------------|
| A. (i) and (iii) are true | B. (i) and (ii) are true |
| C. (iii) and (iv) are true | D. (ii) and (iv) are true |

gate1996 data-structures easy queue stack linked-list

Answer key

3.12.4 Queue: GATE CSE 2001 | Question: 2.16



What is the minimum number of stacks of size n required to implement a queue of size n ?

- | | | | |
|--------|--------|----------|---------|
| A. One | B. Two | C. Three | D. Four |
|--------|--------|----------|---------|

gatecse-2001 data-structures easy stack queue

Answer key

3.12.5 Queue: GATE CSE 2006 | Question: 49



An implementation of a queue Q , using two stacks $S1$ and $S2$, is given below:

```
void insert (Q, x) {  
    push (S1, x);  
}  
void delete (Q) {  
    if (stack-empty(S2)) then
```

```

if (stack-empty(S1)) then {
    print("Q is empty");
    return;
}
else while (!(stack-empty(S1))) {
    x=pop(S1);
    push(S2,x);
}
x=pop(S2);
}

```

Let n insert and $m(\leq n)$ delete operations be performed in an arbitrary order on an empty queue Q . Let x and y be the number of *push* and *pop* operations performed respectively in the process. Which one of the following is true for all m and n ?

- A. $n + m \leq x < 2n$ and $2m \leq y \leq n + m$
- B. $n + m \leq x < 2n$ and $2m \leq y \leq 2n$
- C. $2m \leq x < 2n$ and $2m \leq y \leq n + m$
- D. $2m \leq x < 2n$ and $2m \leq y \leq 2n$

gatecse-2006 data-structures queue stack normal

Answer key

3.12.6 Queue: GATE CSE 2012 | Question: 35



Suppose a circular queue of capacity $(n - 1)$ elements is implemented with an array of n elements. Assume that the insertion and deletion operations are carried out using **REAR** and **FRONT** as array index variables, respectively. Initially, $\text{REAR} = \text{FRONT} = 0$. The conditions to detect queue full and queue empty are:

- A. full : $(\text{REAR} + 1) \bmod n == \text{FRONT}$
empty : $\text{REAR} == \text{FRONT}$
- B. full : $(\text{REAR} + 1) \bmod n == \text{FRONT}$
empty : $(\text{FRONT} + 1) \bmod n == \text{REAR}$
- C. full : $\text{REAR} == \text{FRONT}$
empty : $(\text{REAR} + 1) \bmod n == \text{FRONT}$
- D. full : $(\text{FRONT} + 1) \bmod n == \text{REAR}$
empty : $\text{REAR} == \text{FRONT}$

gatecse-2012 data-structures queue normal

Answer key

3.12.7 Queue: GATE CSE 2013 | Question: 44



Consider the following operation along with Enqueue and Dequeue operations on queues, where k is a global parameter.

```

MultiDequeue(Q){
    m = k
    while (Q is not empty) and (m > 0) {
        Dequeue(Q)
        m = m - 1
    }
}

```

What is the worst case time complexity of a sequence of n queue operations on an initially empty queue?

- A. $\Theta(n)$
- B. $\Theta(n + k)$
- C. $\Theta(nk)$
- D. $\Theta(n^2)$

gatecse-2013 data-structures algorithms normal queue

Answer key

3.12.8 Queue: GATE CSE 2016 | Set 1 | Question: 10



A queue is implemented using an array such that **ENQUEUE** and **DEQUEUE** operations are performed efficiently. Which one of the following statements is **CORRECT** (n refers to the number of items in the

A. Both operations can be performed in $O(1)$ time.
 B. At most one operation can be performed in $O(1)$ time but the worst case time for the operation will be $\Omega(n)$.
 C. The worst case time complexity for both operations will be $\Omega(n)$.
 D. Worst case time complexity for both operations will be $\Omega(\log n)$

Answer key


```
while Q is not Empty do
  if S is Empty OR  $\text{Top}(S) \leq \text{Head}(Q)$  then
    x := Dequeue(Q);
    Push(S, x);
  else
    x := Pop(S);
    Enqueue(Q, x);
  end
end
```

Answer key

Answer key

Diagram illustrating a linked list structure. The list consists of nodes, each represented as a box divided into two parts. The first part of the first node is labeled *head* with an upward arrow. The second part of the first node points to the first part of the second node. This continues through an ellipsis to the last node, whose second part is labeled *tail* with an upward arrow and points to a ground symbol.

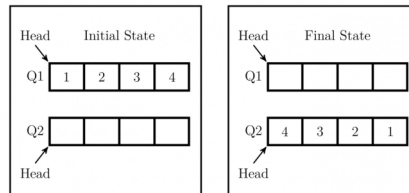
gategse-2018 algorithms data-structures queue normal linked-list one-mark

Answer key

3.12.12 Queue: GATE CSE 2022 | Question: 52



Consider the queues Q_1 containing four elements and Q_2 containing none (shown as the **Initial State** in the figure). The only operations allowed on these two queues are **Enqueue** (Q , element) and **Dequeue** (Q). The minimum number of **Enqueue** operations on Q_1 required to place the elements of Q_1 in Q_2 in reverse order (shown as the **Final State** in the figure) without using any additional storage is _____.



gatecse-2022 numerical-answers data-structures queue two-marks

Answer key

3.12.13 Queue: GATE CSE 2026 | Set 2 | Question: 40



Consider a stack S and a queue Q . Both of them are initially empty and have the capacity to store ten elements each. The elements 1, 2, 3, 4, and 5 arrive one by one, in that order. When an element arrives, it is assigned either to S (pushed on S) or to Q (enqueued to Q). Once all the five elements are stored, the output is generated in two steps. First, stack S is emptied by popping all elements. Then queue Q is emptied by dequeuing all elements. The output obtained by following this process is 43125.

Given the output, the objective is to predict whether an element was assigned to S or Q .

Which of the following options is/are possible valid assignment(s) of the elements?

Note: In the options, the notation xS denotes that element x was assigned to S and yQ denotes that element y was assigned to Q .

- A. 1S, 2Q, 3S, 4S, 5Q
B. 1Q, 2Q, 3S, 4S, 5Q
C. 1Q, 2Q, 3Q, 4S, 5S
D. 1S, 2S, 3S, 4Q, 5Q

gatecse-2026-set2 data-structures stack queue multiple-selects two-marks

Answer key

3.12.14 Queue: GATE DS&AI 2024 | Question: 22



The fundamental operations in a double-ended queue D are:

insertFirst (e) - Insert a new element e at the beginning of D .

insertLast (e) - Insert a new element e at the end of D .

removeFirst () - Remove and return the first element of D .

removeLast () - Remove and return the last element of D .

In an empty double-ended queue, the following operations are performed:

insertFirst (10)

insertLast (32)

$a \leftarrow \text{removeFirst}()$

insertLast (28)

insertLast (17)

$a \leftarrow \text{removeFirst}()$

$a \leftarrow \text{removeLast}()$

The value of a is _____.

Answer key

3.12.15 Queue: GATE IT 2007 | Question: 30



Suppose you are given an implementation of a queue of integers. The operations that can be performed on the queue are:

- `isEmpty(Q)` — returns true if the queue is empty, false otherwise.
- `delete(Q)` — deletes the element at the front of the queue and returns its value.
- `insert(Q, i)` — inserts the integer i at the rear of the queue.

Consider the following function:

```
void f (queue Q) {
    int i ;
    if (!isEmpty(Q)) {
        i = delete(Q);
        f(Q);
        insert(Q, i);
    }
}
```

What operation is performed by the above function f ?

- Leaves the queue Q unchanged
- Reverses the order of the elements in the queue Q
- Deletes the element at the front of the queue Q and inserts it at the rear keeping the other elements in the same order
- Empties the queue Q

Answer key

3.13

Stack (19)

Practice Tests: [Test 1 \(15Q\)](#) [Test 2 \(10Q\)](#)

3.13.1 Stack: GATE CSE 1989 | Question: 4-ii



Compute the postfix equivalent of the following infix arithmetic expression

$$a + b * c + d * e \uparrow f$$

where $\uparrow$ represents exponentiation. Assume normal operator precedences.

Answer key

3.13.2 Stack: GATE CSE 1991 | Question: 03,vii



The following sequence of operations is performed on a stack:

PUSH(10), PUSH(20), POP, PUSH(10), PUSH(20), POP, POP, POP, PUSH(20), POP

The sequence of values popped out is

- 20, 10, 20, 10, 20
- 20, 20, 10, 10, 20
- 10, 20, 20, 10, 20
- 20, 20, 10, 20, 10

Answer key

3.13.3 Stack: GATE CSE 1994 | Question: 1.14



Which of the following permutations can be obtained in the output (in the same order) using a stack assuming that the input is the sequence 1, 2, 3, 4, 5 in that order?

- A. 3, 4, 5, 1, 2
 B. 3, 4, 5, 2, 1
 C. 1, 5, 2, 3, 4
 D. 5, 4, 3, 1, 2

gate1994 data-structures stack normal

Answer key

3.13.4 Stack: GATE CSE 1995 | Question: 2.21



The postfix expression for the infix expression $A + B * (C + D) / F + D * E$ is:

- A. $AB + CD + *F / D + E*$
 B. $ABCD + *F / DE * ++$
 C. $A * B + CD / F * DE ++$
 D. $A + *BCD / F * DE ++$

gate1995 data-structures stack easy

Answer key

3.13.5 Stack: GATE CSE 2000 | Question: 13



Suppose a stack implementation supports, in addition to PUSH and POP, an operation REVERSE, which reverses the order of the elements on the stack.

- A. To implement a queue using the above stack implementation, show how to implement ENQUEUE using a single operation and DEQUEUE using a sequence of 3 operations.
 B. The following post fix expression, containing single digit operands and arithmetic operators $+$ and $*$, is evaluated using a stack.
 $5\ 2\ *\ 3\ 4\ +\ 5\ 2\ * \ +$
 Show the contents of the stack
 i. After evaluating $5\ 2\ * \ 3\ 4\ +$
 ii. After evaluating $5\ 2\ * \ 3\ 4\ +\ 5\ 2$
 iii. At the end of evaluation

gatecse-2000 data-structures stack normal descriptive

Answer key

3.13.6 Stack: GATE CSE 2003 | Question: 64



Let S be a stack of size $n \geq 1$. Starting with the empty stack, suppose we push the first n natural numbers in sequence, and then perform n pop operations. Assume that Push and Pop operations take X seconds each, and Y seconds elapse between the end of one such stack operation and the start of the next operation. For $m \geq 1$, define the stack-life of m as the time elapsed from the end of $Push(m)$ to the start of the pop operation that removes m from S . The average stack-life of an element of this stack is

- A. $n(X + Y)$
 B. $3Y + 2X$
 C. $n(X + Y) - X$
 D. $Y + 2X$

gatecse-2003 data-structures stack normal

Answer key

3.13.7 Stack: GATE CSE 2004 | Question: 3



A single array $A[1 \dots \text{MAXSIZE}]$ is used to implement two stacks. The two stacks grow from opposite ends of the array. Variables $top1$ and $top2$ ($top1 < top2$) point to the location of the topmost element in each of the stacks. If the space is to be used efficiently, the condition for “stack full” is

- A. $(top1 = \text{MAXSIZE}/2)$ and $(top2 = \text{MAXSIZE}/2 + 1)$
 B. $top1 + top2 = \text{MAXSIZE}$
 C. $(top1 = \text{MAXSIZE}/2)$ or $(top2 = \text{MAXSIZE})$
 D. $top1 = top2 - 1$

gatecse-2004 data-structures stack easy

Answer key

3.13.8 Stack: GATE CSE 2004 | Question: 5



The best data structure to check whether an arithmetic expression has balanced parentheses is a

- A. queue B. stack C. tree D. list

gatecse-2004 data-structures easy stack

Answer key

3.13.9 Stack: GATE CSE 2014 | Set 2 | Question: 41



Suppose a stack implementation supports an instruction **REVERSE**, which reverses the order of elements on the stack, in addition to the **PUSH** and **POP** instructions. Which one of the following statements is **TRUE** (*with respect to this modified stack*)?

- A. A queue cannot be implemented using this stack.
B. A queue can be implemented where **ENQUEUE** takes a single instruction and **DEQUEUE** takes a sequence of two instructions.
C. A queue can be implemented where **ENQUEUE** takes a sequence of three instructions and **DEQUEUE** takes a single instruction.
D. A queue can be implemented where both **ENQUEUE** and **DEQUEUE** take a single instruction each.

gatecse-2014-set2 data-structures stack easy

Answer key

3.13.10 Stack: GATE CSE 2015 | Set 2 | Question: 38



Consider the C program below

```
#include <stdio.h>
int *A, stkTop;
int stkFunc (int opcode, int val)
{
    static int size=0, stkTop=0;
    switch (opcode) {
        case -1: size = val; break;
        case 0: if (stkTop < size ) A[stkTop++]=val; break;
        default: if (stkTop) return A[--stkTop];
    }
    return -1;
}
int main()
{
    int B[20]; A=B; stkTop = -1;
    stkFunc (-1, 10);
    stkFunc (0, 5);
    stkFunc (0, 10);
    printf ("%d\n", stkFunc(1, 0)+ stkFunc(1, 0));
}
```

The value printed by the above program is _____.

gatecse-2015-set2 data-structures stack easy numerical-answers

Answer key

3.13.11 Stack: GATE CSE 2015 | Set 3 | Question: 12



The result evaluating the postfix expression $10\ 5 +\ 60\ 6 / * 8 -$ is

- A. 284 B. 213 C. 142 D. 71

gatecse-2015-set3 data-structures stack easy

Answer key

3.13.12 Stack: GATE CSE 2021 | Set 1 | Question: 21



Consider the following sequence of operations on an empty stack.

`push(54); push(52); pop(); push(55); push(62); s = pop();`

Consider the following sequence of operations on an empty queue.

`enqueue(21); enqueue(24); dequeue(); enqueue(28); enqueue(32); q = dequeue();`

The value of $s+q$ is _____.

gatecse-2021-set1 data-structures stack easy numerical-answers one-mark

Answer key

3.13.13 Stack: GATE CSE 2023 | Question: 49



Consider a sequence a of elements $a_0 = 1, a_1 = 5, a_2 = 7, a_3 = 8, a_4 = 9$, and $a_5 = 2$. The following operations are performed on a stack S and a queue Q , both of which are initially empty.

- I. **push** the elements of a from a_0 to a_5 in that order into S .
- II. **enqueue** the elements of a from a_0 to a_5 in that order into Q .
- III. **pop** an element from S .
- IV. **dequeue** an element from Q .
- V. **pop** an element from S .
- VI. **dequeue** an element from Q .
- VII. **dequeue** an element from Q and push the same element into S .
- VIII. Repeat operation VII three times.
- IX. **pop** an element from S .
- X. **pop** an element from S .

The top element of S after executing the above operations is _____.

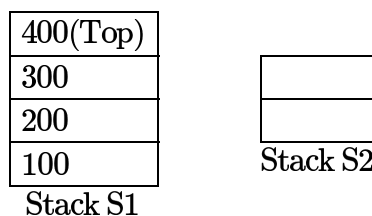
gatecse-2023 data-structures stack numerical-answers two-marks easy

Answer key

3.13.14 Stack: GATE CSE 2024 | Set 2 | Question: 38



Let $S1$ and $S2$ be two stacks. $S1$ has capacity of 4 elements. $S2$ has capacity of 2 elements. $S1$ already has 4 elements: 100, 200, 300, and 400, whereas $S2$ is empty, as shown below.



Only the following three operations are available:

- PushToS2: Pop the top element from $S1$ and push it on $S2$.
- PushToS1: Pop the top element from $S2$ and push it on $S1$.
- GenerateOutput: Pop the top element from $S1$ and output it to the user.

Note that the pop operation is not allowed on an empty stack and the push operation is not allowed on a full stack.

Which of the following output sequences can be generated by using the above operations?

- | | |
|-----------------------|-----------------------|
| A. 100, 200, 400, 300 | B. 200, 300, 400, 100 |
| C. 400, 200, 100, 300 | D. 300, 200, 400, 100 |

gatecse-2024-set2 data-structures stack multiple-selects two-marks

Answer key

3.13.15 Stack: GATE CSE 2025 | Set 2 | Question: 35



Consider a stack data structure into which we can PUSH and POP records. Assume that each record pushed in the stack has a positive integer key and that all keys are distinct.

We wish to augment the stack data structure with an $O(1)$ time MIN operation that returns a pointer to the record with smallest key present in the stack

1. without deleting the corresponding record, and
2. without increasing the complexities of the standard stack operations.

Which one or more of the following approach(es) can achieve it?

- A. Keep with every record in the stack, a pointer to the record with the smallest key below it.
- B. Keep a pointer to the record with the smallest key in the stack.
- C. Keep an auxiliary array in which the key values of the records in the stack are maintained in sorted order.
- D. Keep a Min-Heap in which the key values of the records in the stack are maintained.

gatecse2025-set2 data-structures stack multiple-selects two-marks

Answer key

3.13.16 Stack: GATE DA 2025 | Question: 54



Consider the following pseudocode.

```
Create empty stack S
Set x=0, flag=0, sum=0
Push x onto S
while (S is not empty){
  if (flag equals 0){
    Set x = x+1
    Push x onto S
  }
  if (x equals 8):
    Set flag=1
  if (flag equals 1){
    x = Pop(S)
    if (x is odd):
      Pop (S)
    Set sum = sum + x
  }
}
Output sum
```

The value of **sum** output by a program executing the above pseudocode is _____ (Answer in integer)

gateda-2025 data-structures stack output numerical-answers two-marks

Answer key

3.13.17 Stack: GATE IT 2004 | Question: 52



A program attempts to generate as many permutations as possible of the string, 'abcd' by pushing the characters *a, b, c, d* in the same order onto a stack, but it may pop off the top character at any time. Which one of the following strings CANNOT be generated using this program?

- A. *abcd*
- B. *dcba*
- C. *cbad*
- D. *cabd*

gateit-2004 data-structures normal stack

Answer key

3.13.18 Stack: GATE IT 2005 | Question: 13



A function f defined on stacks of integers satisfies the following properties. $f(\emptyset) = 0$ and $f(\text{push}(S, i)) = \max(f(S), 0) + i$ for all stacks S and integers i .

If a stack S contains the integers 2, -3, 2, -1, 2 in order from bottom to top, what is $f(S)$?

- A. 6
- B. 4
- C. 3
- D. 2

Answer key

3.13.19 Stack: GATE IT 2007 | Question: 32



Consider the following C program:

```
#include <stdio.h>
#define EOF -1
void push (int); /* push the argument on the stack */
int pop (void); /* pop the top of the stack */
void flagError ();
int main ()
{
    int c, m, n, r;
    while ((c = getchar ()) != EOF)
    {
        if (isdigit (c))
            push (c);
        else if ((c == '+') || (c == '*'))
        {
            m = pop ();
            n = pop ();
            r = (c == '+') ? n + m : n*m;
            push (r);
        }
        else if (c != ' ')
            flagError ();
    }
    printf("%c", pop ());
}
```

What is the output of the program for the following input?

5 2 * 3 3 2 + * +

- A. 15 B. 25 C. 30 D. 150

Answer key

3.14

Time Complexity (1)

3.14.1 Time Complexity: GATE CSE 2025 | Set 2 | Question: 28



A meld operation on two instances of a data structure combines them into one single instance of the same data structure. Consider the following data structures:

- P. Unsorted doubly linked list with pointers to the head node and tail node of the list.
 Q. Min-heap implemented using an array.
 R. Binary Search Tree.

Which ONE of the following options gives the worst-case time complexities for meld operation on instances of size n of these data structures?

- A. P : $\Theta(1)$, Q : $\Theta(n)$, R : $\Theta(n)$
 B. P : $\Theta(1)$, Q : $\Theta(n \log n)$, R : $\Theta(n)$
 C. P : $\Theta(n)$, Q : $\Theta(n \log n)$, R : $\Theta(n^2)$
 D. P : $\Theta(1)$, Q : $\Theta(n)$, R : $\Theta(n \log n)$

Answer key

3.15

Tree (13)

Practice Tests: Test 1 (15Q) Test 2 (3Q)

3.15.1 Tree: GATE CSE 1990 | Question: 13a



Consider the height-balanced tree T_t with values stored at only the leaf nodes, shown in Fig.4.

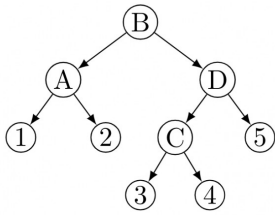


Fig.4

(i) Show how to merge to the tree, T_1 elements from tree T_2 shown in Fig.5 using node D of tree T_1 .

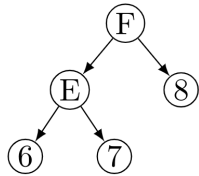


Fig.5

(ii) What is the time complexity of a merge operation of balanced trees T_1 and T_2 where T_1 and T_2 are of height h_1 and h_2 respectively, assuming that rotation schemes are given. Give reasons.

gate1990 data-structures tree descriptive

Answer key

3.15.2 Tree: GATE CSE 1992 | Question: 02,vii



A 2 – 3 tree is such that

- All internal nodes have either 2 or 3 children
- All paths from root to the leaves have the same length

The number of internal nodes of a 2 – 3 tree having 9 leaves could be

- 4
- 5
- 6
- 7

gate1992 tree data-structures normal multiple-selects

Answer key

3.15.3 Tree: GATE CSE 1994 | Question: 5



A 3 – ary tree is a tree in which every internal node has exactly three children. Use induction to prove that the number of leaves in a 3 – ary tree with n internal nodes is $2(n + 1)$.

gate1994 data-structures tree proof descriptive

Answer key

3.15.4 Tree: GATE CSE 1998 | Question: 1.24



Which of the following statements is false?

- A tree with a n nodes has $(n - 1)$ edges
- A labeled rooted binary tree can be uniquely constructed given its postorder and preorder traversal results.
- A complete binary tree with n internal nodes has $(n + 1)$ leaves.
- The maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$

gate1998 data-structures tree multiple-selects normal

Answer key

3.15.5 Tree: GATE CSE 1998 | Question: 2.11



A complete n -ary tree is one in which every node has 0 or n sons. If x is the number of internal nodes of a complete n -ary tree, the number of leaves in it is given by

- A. $x(n-1) + 1$ B. $xn - 1$ C. $xn + 1$ D. $x(n+1)$

gate1998 data-structures tree normal

Answer key

3.15.6 Tree: GATE CSE 2002 | Question: 2.9



The number of leaf nodes in a rooted tree of n nodes, with each node having 0 or 3 children is:

- A. $\frac{n}{2}$ B. $\frac{(n-1)}{3}$ C. $\frac{(n-1)}{2}$ D. $\frac{(2n+1)}{3}$

gatecse-2002 data-structures tree normal

Answer key

3.15.7 Tree: GATE CSE 2004 | Question: 6



Level order traversal of a rooted tree can be done by starting from the root and performing

- A. preorder traversal B. in-order traversal
C. depth first search D. breadth first search

gatecse-2004 data-structures tree easy

Answer key

3.15.8 Tree: GATE CSE 2005 | Question: 36



In a complete k -ary tree, every internal node has exactly k children. The number of leaves in such a tree with n internal node is:

- A. nk B. $(n-1)k + 1$ C. $n(k-1) + 1$ D. $n(k-1)$

gatecse-2005 data-structures tree normal

Answer key

3.15.9 Tree: GATE CSE 2007 | Question: 43



A complete n -ary tree is a tree in which each node has n children or no children. Let I be the number of internal nodes and L be the number of leaves in a complete n -ary tree. If $L = 41$ and $I = 10$, what is the value of n ?

- A. 3 B. 4 C. 5 D. 6

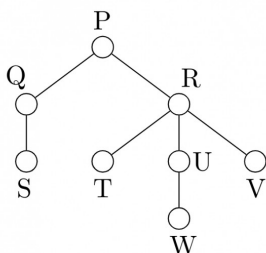
gatecse-2007 data-structures tree normal

Answer key

3.15.10 Tree: GATE CSE 2014 | Set 3 | Question: 12



Consider the following rooted tree with the vertex labeled P as the root:



The order in which the nodes are visited during an in-order traversal of the tree is

- A. SQPTRWUV B. SQPTUWRV C. SQPTWUVR D. SQPTRUWV

gatecse-2014-set3 data-structures tree easy

Answer key

3.15.11 Tree: GATE CSE 2014 | Set 3 | Question: 41



Consider the pseudocode given below. The function *DoSomething()* takes as argument a pointer to the root of an arbitrary tree represented by the *leftMostChild – rightSibling* representation. Each node of the tree is of type *treeNode*.

```
typedef struct treeNode* treeptr;

struct treeNode
{
    treeptr leftMostChild, rightSibling;
};

int DoSomething (treeptr tree)
{
    int value=0;
    if (tree != NULL) {
        if (tree->leftMostChild == NULL)
            value = 1;
        else
            value = DoSomething(tree->leftMostChild);
        value = value + DoSomething(tree->rightSibling);
    }
    return(value);
}
```

When the pointer to the root of a tree is passed as the argument to *DoSomething*, the value returned by the function corresponds to the

- A. number of internal nodes in the tree. B. height of the tree.
C. number of nodes without a right sibling in the tree. D. number of leaf nodes in the tree

gatecse-2014-set3 data-structures tree normal

Answer key

3.15.12 Tree: GATE CSE 2017 | Set 1 | Question: 20



Let T be a tree with 10 vertices. The sum of the degrees of all the vertices in T is _____

gatecse-2017-set1 data-structures tree easy numerical-answers

Answer key

3.15.13 Tree: GATE CSE 2021 | Set 1 | Question: 41



An *articulation point* in a connected graph is a vertex such that removing the vertex and its incident edges disconnects the graph into two or more connected components.

Let T be a DFS tree obtained by doing DFS in a connected undirected graph G .

Which of the following options is/are correct?

- A. Root of T can never be an articulation point in G .
B. Root of T is an articulation point in G if and only if it has 2 or more children.
C. A leaf of T can be an articulation point in G .
D. If u is an articulation point in G such that x is an ancestor of u in T and y is a descendent of u in T , then all paths from x to y in G must pass through u .

gatecse-2021-set1 multiple-selects data-structures tree two-marks

Answer key

3.16.1 Tree Traversal: GATE CSE 2026 | Set 2 | Question: 2



The set **T** represents various traversals over binary tree. The set **S** represents the order of visiting nodes during a traversal.

T	S
I: Inorder	L: left subtree, node, right subtree
II: Preorder	M: node, left subtree, right subtree
III: Postorder	N: left subtree, right subtree, node

Which one of the following is the correct match from **T** to **S** ?

- A. I – L, II – M, III – N
 B. I – M, II – L, III – N
 C. I – N, II – M, III – L
 D. I – L, II – N, III – M

gatecse-2026-set2 data-structures tree-traversal one-mark

Answer key

3.17.1 Uniform Hashing: GATE DS&AI 2024 | Question: 11



Consider performing uniform hashing on an open address hash table with load factor $\alpha = \frac{n}{m} < 1$, where n elements are stored in the table with m slots. The expected number of probes in an unsuccessful search is at most $\frac{1}{1-\alpha}$.

Inserting an element in this hash table requires at most probes, _____ on average.

- A. $\ln\left(\frac{1}{1-\alpha}\right)$ B. $\frac{1}{1-\alpha}$ C. $1 + \frac{\alpha}{2}$ D. $\frac{1}{1+\alpha}$

gate-ds-ai-2024 data-structures hashing uniform-hashing one-mark

Answer key

Answer Keys

3.1.1	N/A	3.1.2	B	3.1.3	N/A	3.1.4	B	3.1.5	C
3.1.6	A	3.2.1	C	3.3.1	N/A	3.3.2	C	3.3.3	N/A
3.3.4	N/A	3.3.5	A	3.3.6	A	3.3.7	N/A	3.3.8	A
3.3.9	B	3.3.10	C	3.3.11	B	3.3.12	5	3.3.13	C
3.4.1	N/A	3.4.2	C	3.4.3	N/A	3.4.4	B	3.4.5	D
3.4.6	A	3.4.7	D	3.4.8	A	3.4.9	D	3.4.10	A
3.4.11	B	3.4.12	C	3.4.13	D	3.4.14	B	3.4.15	A
3.4.16	B	3.4.17	A	3.4.18	D	3.4.19	B	3.4.20	8
3.4.21	80	3.4.22	A	3.4.23	511	3.4.24	C	3.4.25	B
3.4.26	A	3.4.27	5:5	3.4.28	B	3.4.29	C	3.4.30	B
3.5.1	B	3.5.2	N/A	3.5.3	D	3.5.4	N/A	3.5.5	C
3.5.6	B	3.5.7	B	3.5.8	B	3.5.9	B	3.5.10	A

3.5.11	B
3.5.16	110
3.5.21	B
3.5.26	509
3.5.31	D
3.5.36	B
3.6.5	N/A
3.6.10	N/A
3.6.15	B
3.6.20	C
3.6.25	D
3.6.30	C
3.6.35	A
3.6.40	4.25
3.6.45	A;B;C
3.6.50	B
3.7.2	B
3.8.3	N/A
3.8.8	C
3.8.13	C
3.9.3	A
3.10.4	B
3.10.9	N/A
3.10.14	B
3.10.19	A
3.10.24	B
3.12.3	A
3.12.8	A
3.12.13	A;B
3.13.3	B
3.13.8	B
3.13.13	8
3.13.18	C
3.15.3	N/A
3.15.8	C
3.15.13	B

3.5.12	B
3.5.17	A
3.5.22	B
3.5.27	A
3.5.32	C
3.6.1	False
3.6.6	A;C;D
3.6.11	N/A
3.6.16	N/A
3.6.21	D
3.6.26	D
3.6.31	A
3.6.36	19
3.6.41	1 : 1
3.6.46	D
3.6.51	D
3.7.3	12
3.8.4	C
3.8.9	A
3.8.14	B
3.9.4	A
3.10.5	D
3.10.10	D
3.10.15	D
3.10.20	D
3.11.1	D
3.12.4	B
3.12.9	256
3.12.14	17
3.13.4	B
3.13.9	C
3.13.14	B;C;D
3.13.19	B
3.15.4	B;C
3.15.9	C
3.16.1	A

3.5.13	C
3.5.18	B
3.5.23	B
3.5.28	A;B
3.5.33	A
3.6.2	False
3.6.7	B
3.6.12	N/A
3.6.17	C
3.6.22	D
3.6.27	C
3.6.32	B
3.6.37	199
3.6.42	C
3.6.47	B
3.6.52	B
3.7.4	A
3.8.5	B
3.8.10	A
3.8.15	D
3.10.1	B
3.10.6	C
3.10.11	B
3.10.16	C
3.10.21	5:5
3.11.2	B
3.12.5	A
3.12.10	B
3.12.15	B
3.13.5	N/A
3.13.10	15
3.13.15	A
3.14.1	A
3.15.5	A
3.15.10	A
3.17.1	B

3.5.14	D
3.5.19	B
3.5.24	B
3.5.29	4:4
3.5.34	D
3.6.3	N/A
3.6.8	144
3.6.13	N/A
3.6.18	N/A
3.6.23	N/A
3.6.28	B
3.6.33	A
3.6.38	C
3.6.43	B
3.6.48	D
3.6.53	A
3.8.1	C
3.8.6	C
3.8.11	B
3.9.1	A
3.10.2	N/A
3.10.7	N/A
3.10.12	A
3.10.17	B
3.10.22	B
3.12.1	N/A
3.12.6	A
3.12.11	B
3.13.1	N/A
3.13.6	C
3.13.11	C
3.13.16	24:24
3.15.1	N/A
3.15.6	D
3.15.11	D

3.5.15	C
3.5.20	64
3.5.25	D
3.5.30	C
3.5.35	C
3.6.4	N/A
3.6.9	N/A
3.6.14	N/A
3.6.19	N/A
3.6.24	B
3.6.29	A
3.6.34	1
3.6.39	4
3.6.44	A;B;C
3.6.49	C
3.7.1	B
3.8.2	N/A
3.8.7	C
3.8.12	80
3.9.2	N/A
3.10.3	N/A
3.10.8	N/A
3.10.13	D
3.10.18	C
3.10.23	A
3.12.2	N/A
3.12.7	A
3.12.12	0
3.13.2	B
3.13.7	D
3.13.12	86 : 86
3.13.17	D
3.15.2	A;D
3.15.7	D
3.15.12	18